



Universidad de Murcia

Facultad de Informática

TRABAJO FIN DE GRADO

Módulo en Python para gestionar asociaciones de seguridad IPsec usando SDN e IKE en dispositivos de red

Autor

José Padilla Martínez

Directores

Rafael Marín López

Gabriel López Millán

MURCIA, A 24 DE JUNIO DE 2020

De acuerdo al reglamento que regula los trabajos de fin de grado en la Facultad de Informática de la Universidad de Murcia, declaro que asumo la originalidad del trabajo presentado y que todas las fuentes utilizadas han sido debidamente citadas.

Fdo: José Padilla Martínez.

A handwritten signature in black ink, appearing to read 'José Padilla', with a long horizontal stroke extending to the right.

Murcia, a día 24 de junio de 2020.

Índice

Resumen	9
Extended abstract	11
1. Introducción	17
2. Estado del arte	21
2.1. SDN (<i>Software Defined Networking</i>)	21
2.1.1. Motivación y descripción.	21
2.1.2. Arquitectura	21
2.1.3. Funcionamiento y caso de uso	24
2.2. NETCONF (<i>Network Configuration Protocol</i>)	26
2.2.1. Motivación y descripción	26
2.2.2. Arquitectura	26
2.2.3. Operaciones	28
2.3. IPsec (<i>Internet Protocol Security</i>)	30
2.3.1. Motivación y descripción	30
2.3.2. Arquitectura y procesamiento	30
2.3.3. Modos de funcionamiento	34
2.4. IKE (<i>Internet Key Exchange</i>)	37
2.4.1. Motivación y descripción	37
2.4.2. Arquitectura y funcionamiento	37
2.5. Implementaciones <i>OpenSource</i> de IKE	41
2.6. VICI (<i>Versatile IKE Configuration Interface</i>)	42
2.6.1. Motivación y descripción	42
2.6.2. Arquitectura y funcionamiento	42
2.6.3. Comandos	44
2.7. Contextualización del trabajo	46
2.7.1. Vías de desarrollo	47
2.7.2. Escenario desarrollado	48
3. Objetivos y metodología	53
4. Diseño de la solución	57
4.1. Administración con IKE en el dispositivo	57
4.1.1. Establecimiento de canal seguro	57
4.1.2. Funcionamiento interno del establecimiento	59
4.2. Funcionamiento interno del dispositivo de red	62
4.2.1. <i>Sysrepo</i>	62
4.2.2. <i>Netopeer2</i>	63
4.2.3. Procesamiento interno de configuración	63
4.3. Módulo <i>ietf-ipsec.py</i>	65

4.3.1. Procesamiento de configuración recibida	65
4.3.2. Estructuras internas y atributos	67
5. Despliegue del escenario y pruebas	71
5.1. Despliegue del escenario	71
5.2. Prueba final de funcionamiento	73
5.2.1. Prueba de funcionamiento en modo transporte	73
5.2.2. Prueba de funcionamiento en modo túnel	75
5.2.3. Contenido de estructuras internas	78
6. Conclusiones y vías futuras	81
Bibliografía	83
Anexo A - Puesta en marcha del escenario	85
Anexo B - Ficheros XML utilizados	93

Índice de figuras

1.	Arquitectura general de SDN.	22
2.	Funcionamiento de SDN.	24
3.	Arquitectura general de NETCONF [10].	27
4.	Procedimiento al invocar a IPsec, flujo entrante y saliente [19]. . . .	31
5.	Administración de IPsec, flujo saliente [20].	33
6.	Administración de IPsec, flujo entrante [20].	34
7.	Funcionamiento de modo túnel [22].	35
8.	Funcionamiento de modo transporte [22].	35
9.	Cabeceras AH, modo túnel vs modo transporte.	36
10.	Cabeceras ESP, modo túnel vs modo transporte.	36
11.	Negociación inicial de IKE.	38
12.	Negociación de una asociación de seguridad de IKE.	39
13.	Aplicación de asociaciones de seguridad.	48
14.	Topología del escenario <i>host-to-host</i>	49
15.	Topología del escenario <i>gateway-to-gateway</i>	50
16.	Topología del escenario <i>n-hosts</i>	51
17.	Flujo de mensajes en escenario <i>host-to-host</i>	58
18.	Funcionamiento interno del establecimiento de asociaciones.	60
19.	Funcionamiento interno del servidor NETCONF.	62
20.	Topología final del escenario del caso de uso.	71
21.	Información en <i>host 1</i> una vez aplicada la configuración (transporte). .	74
22.	Información en <i>host 2</i> una vez aplicada la configuración (transporte). .	75
23.	Negociación IKE y paquetes ICMP protegidos en modo transporte. . .	75
24.	Información en <i>host 1</i> una vez aplicada la configuración (túnel). . .	76
25.	Información en <i>host 1</i> una vez aplicada la configuración (túnel). . .	77
26.	Negociación IKE y paquetes ICMP protegidos en modo túnel.	77

Resumen

El desarrollo de este trabajo de fin de grado se basa en la idea de mejorar la pobre optimización de la arquitectura de red actual, que se ve constantemente degradada por el aumento de dispositivos y servicios cada vez más complejos y con mayores requisitos, provocando así una lenta respuesta a todos ellos y una saturación de la red. Actualmente, podemos observar importantes dificultades en una amplia variedad de aspectos como, por ejemplo, a la hora de intentar implementar una política que englobe a toda la red, lo que provocaría tener que ir actualizando todos los dispositivos de la red uno a uno, pudiendo llevar al administrador de red un tiempo excesivo. Otro problema importante se debe a la complejidad que la gestión de la red posee conforme se van añadiendo nuevos elementos a ella, lo que provoca definir reglas y protocolos diferentes para grupos cerrados de elementos de la red, derivando así la gestión de la red en una tarea ardua y muy poco escalable.

Es en este punto cuando aparece el paradigma SDN (*Software Define Networking*), un concepto que pretende solventar, o al menos minimizar, los problemas más relevantes de la arquitectura de red actual. Este nuevo paradigma intenta separar la administración de la red de la propia infraestructura, consiguiendo así ajustar dinámicamente el flujo de tráfico en toda la red, simplificando en gran parte la respuesta a las necesidades cambiantes de esta. Para conseguirlo, desacopla el plano de control del plano de datos, consiguiendo una diferenciación entre la parte software y hardware. Esta arquitectura de red desemboca en una importante homogeneidad a la hora de configurar la red, lo que aporta una considerable mejora en la gestión y actualización de recursos, junto a una gran escalabilidad y flexibilidad. Aparece en este paradigma la figura del *Controlador*, que será la entidad central de toda la arquitectura, y que se encargará, entre otras muchas tareas, de transmitir la información necesaria a todos los enrutadores y conmutadores, de tal manera que sepan que hacer con el tráfico entrante.

Este paradigma representa el futuro de la arquitectura de redes global, lo que implica necesariamente investigación y avance en todos sus ámbitos. Con el uso del modelo SDN en la arquitectura, se puede conseguir una notable mejora en la gestión de la red, como por ejemplo en aspectos de movilidad o enrutamiento, centrándonos en la seguridad. Aunque las investigaciones hoy día giran en una amplia variedad de campos, como eficiencia, movilidad o escalabilidad, en este documento hacemos especial hincapié en la seguridad, puesto que es algo fundamental hoy en día, pero no se dispone de consenso ni estándares oficiales a la hora de proteger las diferentes comunicaciones que se producen en la arquitectura. No partimos desde cero, puesto que disponemos de material de anteriores trabajos de fin de grados, el cual tenemos accesible y hemos usado como punto de partida.

Actualmente, la gestión de las políticas de seguridad no han sido lo suficiente-

mente optimizas, debido sobre todo a la falta de automatización al aplicarlas. Una vez recibidas las nuevas configuraciones en un dispositivo, este no es capaz de aplicar la configuración al instante, requiriendo la creación o modificación de ficheros de configuración y/o el reinicio de servicios para aplicar los cambios. Partiendo de esta motivación, vamos a hacer uso de una interfaz que nos permita la instanciación de las políticas directamente en el kernel en caliente, llamada VICI (*Versatile IKE Configuration Interface*), simplificando en gran medida la ardua tarea de gestionar la configuración y el control del demonio IKE, llamado *charon* (en nuestro caso hacemos uso de la segunda versión del protocolo, IKEv2). El problema se ve incrementado debido a la complejidad que poseen los módulos que funcionan en los nodos (módulos encargados de procesar e instanciar la configuración recibida), sumado a la implementación que fue realizada en C. Como solución, vamos a desarrollar un nuevo módulo implementado en lenguaje *Python* que nos proporcione la funcionalidad necesaria con un bajo nivel de complejidad, haciendo uso de la interfaz de VICI para una vez procesada la información entrante, poder aplicarla en caliente en el dispositivo.

A lo largo del documento, vamos a mostrar todo el proceso desde que un administrador quiere introducir una nueva asociación de seguridad, hasta el momento en el que se haya actualizado correctamente la asociación y se puedan comunicar de forma segura los diferentes componentes a los que afecte esa asociación. Para ello, simularemos un escenario en el que tendremos un controlador y dos nodos de la red, siendo estos nodos sobre los cuales cargaremos la asociación, comprobando finalmente como se comunican de forma segura mediante IPsec.

Finalmente, a modo de conclusión, destacar que una vez desarrollado este proyecto, quedan diferentes vías de trabajo abiertas para su investigación, como pueden ser una mejora del código del controlador que permita su correcto funcionamiento en escenarios más amplios o el uso de otros protocolos para establecer una comunicación segura, tanto entre controlador y nodos como en entre estos últimos. Sobre la mejora que realizamos en este trabajo, quedan también abiertos diferentes frentes de avance, como por ejemplo un código que nos permita la adición de más atributos en las configuraciones, destacando el uso de AH como modo de IPsec.

Extended abstract

The constant technological advancements in society have led to the globalization of the network and its use for a greater variety of cases. The versatility, mobility, speed or connection capacity at any point that the network offers, among many other advantages, have led to the appearance of new companies, services and devices which make use of the network in all its possibilities, involving a constant increase in network traffic, and consequently, the necessary bandwidth. The large increase in traffic is in great measure due to virtualization, a technique that enables multiple tasks to be performed on the same machine.

All this increase represents a challenge to be overcome by the network which has to be constantly renewed trying to provide quality service. As requirements increase, new generations of communication are developed (as happens with the case for the fifth generation), providing improvements in security, speed or latency, among others, trying to adjust at all times to the new parameters that the network needs. However, the research or progress that has been made in the management of network architecture have not been enough and this will be the main focus along the development of this document.

Nowadays, as we have commented, the network architecture and its management do not have sufficiently optimization, being constantly degraded by the increases in traffic, which leads to the appearance of a series of problems, focusing on those related to management and configuration of network. Among the large number of difficulties that we can currently encounter, we can highlight the implementation of policies that encompass the network or the constant increase in complexity as new elements are added to it. These problems lead to complex and poorly scalable network management, especially by the network administrators themselves. One of the main problems that current architecture presents us with is its static nature, which does not allow adaptation to changes in the network. Another of the main problems is based on the individual functionality of the different components of the network, where each node of the network makes its own decisions without taking into account the rest of the topology, in other words, each node acts independently. This is due to the fact that each node owns both the data plane and the control plane.

To solve this problem, the SDN (Software Defined Networking) paradigm appears, a concept that aims to eliminate the most relevant problems of the current network architecture. We emphasize that the paradigm as such is not something new. In fact, it has been developed for years, but it is now, with the abrupt growth in bandwidth that is demanded by society, when the needs and benefits that its application will bring have become more visible. The use of this paradigm is intended to greatly simplify the tasks related to managing the network infrastructure, allowing global traffic control and dynamic adjustment of network behaviour. The root

of this simplification is based on decoupling the control and data planes, separating the network administration from the network infrastructure itself, thus dynamically adjusting the traffic flow throughout the network, placing great emphasis on the differentiation between the software and hardware part.

The structure of this paradigm will have three different plans, which are the application plane, the control plane and the data plane. On the application side, it aims to abstract the administrator from the complexity of network management, presenting a simple interface for configuring and consulting statistics and information in real-time. The control plane will be made up of one or more entities called controllers, and these will be what has a global view of the architecture and are in charge of managing the different network end nodes. These nodes will be within the data plane, it will be the bottom plane of the topology and its function is to route the traffic following the guidelines indicated by the corresponding controller. This new approach provides great uniformity in the management of the network, providing it with high flexibility and scalability, and eliminating the need that previously had in a large number of cases a physical presence for the management of different nodes, updates or simply maintenance.

Communication between the different planes is done through one or more APIs, which work on different interfaces. Starting from the central controller location, it has a Northbound interface for communication between applications and controllers, a Southbound interface for communication between a controller and the network end nodes, and finally, Eastbound and Westbound interfaces for communication between the different drivers. Within these plans and interfaces, the project will focus on the use of an API in the Southbound interface, studying in depth the communication between a specific controller and each one of the network nodes.

Currently, in this paradigm, progress has been made in a wide variety of fields, such as quality of service or the development of different APIs for communication, but we denote poor security in the global network, and mostly among the nodes themselves. For this reason, we will carry out a more in-depth study on the security of communications between the different nodes, applying different security associations between the nodes of the network and verifying that the communication is secure. At this point, a great advantage of this approach can be seen, since by having a global vision of the network, a controller can simultaneously secure the entire path that a data packet must travel, communicating and instantiating security policies among all route nodes.

For the development of the project, we make use in the first place of an API that works in the Southbound interface that allows us a simple communication between the control and data planes, which means, between the controller and each of the nodes. This API is based on the protocol NETCONF and simplifies communication

for us, allowing us to install, configure and eliminate the different configurations that we apply to each end device. The communication between the controller and each device is done through the use of RPC (Remote Procedure Call), which allows us to execute code on the configured machine. On the other hand, focused on security, we need to select a protocol that satisfies the requirements of our scenario. Since we have decided to secure the traffic at the network level, we need a protocol to provide us with protection at the packet level. Based on previous work, we use IPsec, a protocol that offers us this functionality, together with the main security principles, such as integrity or authentication. For the different nodes of the network to be able to carry out the operation in IPsec, a series of databases are required that will store all the associations related to security, these are SPD, PAD and SAD. It also provides us with two main protocols, which are AH (it provides us with authentication) and ESP (it provides us with authentication and encryption), along with a protocol that delegates the negotiation of keys and the establishment of associations in case of not having any already created among the components of concrete communication, called IKE (in our case IKEv2).

Finally, IPsec also gives us two main patterns of operation, being first the tunnel mode (the mode that protects only between two intermediate nodes of the network, normally between two reliable networks) and secondly the transport mode (the mode that protects end to end of communication). Using IKE eliminates the previous need for manually creating security associations, simplifying administrator management. Finally, we will also make use of an interface that gives us the possibility of loading the configuration more efficiently, called VICI, with which the complicated task of managing configuration and control of IKE will be greatly simplified, offering us the possibility of loading the new configuration from the code itself and automating the process. In the development of the work, a module is created that will be executed in the network device, in charge of processing the received configuration and applying it directly to the device. To apply it, it makes use of the VICI interface. The development of this module is the central pillar on which work research is centered, named *ietf-ipsec.py*.

Throughout the document, we will analyse the current state of the network architecture, together with the motivation for the use of the different technologies and the internal operation of these. We will also investigate the methodology used and the different goals that we set with the completion of the project. In due course, we will first make a theoretical design of the solution that we are going to carry out and then we will show real and functional proof of concept, where the entire internal operation process will be shown since an administrator wants to introduce a new security association between different nodes, until the moment in which the association is applied and configured correctly, point where a secure communication between them can be established. About the operation that this paradigm would have in a real scenario, we will indicate the main steps. First, the administrator needs

to have defined the rules for creating the security associations and entered them in the corresponding controller using an API in the Northbound interface. When the controller has the corresponding information, it can communicate with the different nodes and implement the new security policies, finally leaving the configuration applied.

In the final chapter shown in the document, we will simplify the topology using only one controller and two nodes, where we will manually configure the association, using NETCONF to configure the two nodes. This configuration will be encoded in XML and will be validated against a YANG model. When the different verifications have been passed and the information is correct, the nodes are configured, finally showing how the traffic between them has been secured. In carrying out the test, the change of different parameters will be tested, such as the time required to reload a new association or the operation in tunnel mode or IPsec transport mode. All the development of the scenario will be done simulating the different components in virtual containers using docker tools, and the code used by the devices will be programmed in the Python language.

In first instance, we will have a container that will act as a controller, having NETCONF implemented as a client, and prepared the different configurations related to the different nodes that it has to configure. Furthermore, we will have two containers that will act as network nodes, which will implement NETCONF as servers, to which the controller will connect at the moment that he wants to apply a new policy. These nodes will also execute a code that will receive the different configurations and will instantiate them directly in their nucleus (using VICI in the Python code that we have designed). It is fundamental for these steps, a format of the messages exchanged unified, and obviously, that each one of the nodes can understand and interpret the information received. Lastly, there will be a simulated network between the different nodes and another network between the nodes and the controller itself. In the first network you will see the application of security associations and through the second one, these settings will be applied.

The globalization of this paradigm will mean an improvement both for users and for all companies and services that use the Internet to carry out their activity since it will mean an improvement in efficiency, scalability, mobility and quality of service, among others, especially focused on administration. of the network. This paradigm is the present and, above all, the future of global network architecture, which necessarily implies a constant advance in both research and development in all its characteristics and variants. We have focused especially on security, since there is not a great development because there is no consensus or official standards when it comes to protecting the different communications that occur in the architecture. At the time of developing this project, we did not start from scratch, since we have other projects as a starting point. There are some variants that can be improved or

other avenues of research development, such as an improvement in the module to be able to process associations that make use of other methods (such as AH), or a better implementation that allows reporting changes in the device to the controller.

1. Introducción

Conforme va avanzando la sociedad, el funcionamiento de esta se va centrando cada vez más en el uso de tecnologías que basan su funcionamiento en la red, asociado principalmente a las ventajas que ofrece a las personas, como movilidad o rapidez de la comunicación. Todo esto sumado al notable crecimiento de los servicios virtualizados (permitiendo la ejecución de múltiples dispositivos y servicios en un mismo momento), provoca un crecimiento en la demanda de servicios y desemboca en una necesaria evolución en la red. Actualmente está en auge el desarrollo de la quinta generación de las comunicaciones con el objetivo de solventar la gran demanda de ancho de banda, baja latencia o disponibilidad.

La arquitectura actual de la red posee un gran problema a la hora de gestionar los diferentes dispositivos de la red, puesto que cada uno posee tanto la capacidad de tomar decisiones de enrutamiento como de aplicarlas, es decir, que cada dispositivo posee conjuntamente los planos de control y de datos, complicando la gestión y actuando cada uno de manera individual. En este punto aparece el paradigma SDN (*Software Defined Networking*), que tiene como objetivo permitir un mejor control de los dispositivos de la red, desacoplando los planos de datos y de control, dejando en los dispositivos finales únicamente la capacidad de decisión en función de unas reglas que se encarga de aplicar un controlador central, que posee una vista global de la red y puede tomar decisiones en tiempo real.

Es decir, la tendencia actual se centra en quitar la capacidad de tomar decisiones a los dispositivos finales, con la finalidad de centralizar la toma de decisiones en un único punto y realizar la gestión de la red de una manera más flexible, puesto que una toma de decisiones individual no permite una gestión de los datos eficiente. Al separar estos planos, obtenemos toda la funcionalidad relacionada con control y gestión de la red en un único punto de la red, permitiendo a un administrador o a un módulo automatizado (ambos casos a través del controlador) la capacidad de tomar decisiones de encaminamiento en función de los criterios oportunos. Por ejemplo, si en alguna parte de la red se detecta congestión, se puede indicar a todos los dispositivos que estén rodeando a la zona congestionada que encaminen el tráfico por otras rutas hasta que se haya solucionado la congestión detectada. De esta manera, se simplifica, en gran medida, la gestión global de los dispositivos de la red.

La arquitectura SDN debe permitir en primer lugar una comunicación entre cualquier dispositivo del plano de datos y el controlador, en ambas direcciones, ya sea cualquier dispositivo preguntando acerca del procesamiento de cualquier paquete o el propio controlador instanciando configuraciones o consultando estados de la red. Para ello, en la interfaz *Southbound* (interfaz de comunicación entre controlador y dispositivos) tendremos funcionando un protocolo que nos simplificará y unifica la comunicación entre estos haciendo uso de un intercambio de mensajes definido en

el propio protocolo. A parte de esta interfaz, posee una interfaz *Northbound* con la que se comunican las aplicaciones de alto nivel que maneja un administrador de red y los distintos controladores, y una pareja de interfaces *Eastbound* y *Westbound* que permiten la comunicación entre diferentes controladores.

Conforme se fue avanzando en el desarrollo de este tipo de escenarios, aparecen también diferentes topologías que requieren una forma centralizada de gestionar los aspectos relacionados con la seguridad, para proteger el tráfico de datos entre los dispositivos gestionados. En concreto, hay diferentes escenarios que tienen la necesidad de gestionar las asociaciones de seguridad IPsec. Un ejemplo podrían ser las redes WAN definidas por software (*SD-WAN*), extensión de SDN que proporciona una abstracción de software para crear redes seguras sobre WAN. También aparecen los escenarios relacionados con centros de datos en la nube (*Cloud Data Center Scenario*), donde la administración dinámica de claves es crítica para asegurar la VPN y la distribución de políticas, establecidas usando IPsec [1]. En general, en el crecimiento de redes basadas en SDN, es cada vez más necesario un mecanismo que permita la gestión de las asociaciones de seguridad de forma centralizada.

Al comienzo del desarrollo, se nos presentan dos posibles casos a la hora de la gestión de las asociaciones. Una primera variante se centra en la delegación de la negociación de claves en un protocolo externo al controlador y una segunda variante orientada en la gestión directa desde el controlador, donde este se encarga directamente de gestionar toda la información y la negociación de las claves. El desarrollo de este trabajo se centra en el primer caso, haciendo uso del protocolo IKEv2 para la negociación de claves. Hemos observado la oportunidad de ofrecer una mejora en las implementaciones de los dispositivos, consiguiendo una mayor flexibilidad a la hora de realizar pruebas en los escenarios.

Para ello, se pretende el desarrollo de un módulo que nos permita la gestión de las configuraciones recibidas por un dispositivo desde un controlador. Este módulo estará ubicado en los dispositivos finales de la topología, y se encargará de recibir, procesar y aplicar las configuraciones recibidas, y se llamará *ietf-ipsec.py*. En el punto de partida de este trabajo, ya se tiene un módulo que realiza la misma actividad diseñado en trabajos previos, aunque está programado en C y supone complejidad en el código y en el funcionamiento. Por ello, se plantea la necesidad de volver a implementar el código del módulo en un lenguaje diferente que nos simplifique el funcionamiento y el desarrollo, haciendo uso en nuestro caso de *Python*.

Para la aplicación de la configuración recibida en el módulo, desde el controlador, una vez procesada, hacemos uso de una interfaz llamada VICI (*Versatile IKE Configuration Interface*), que nos permite un control total sobre IKE (usado para la negociación de las claves) y la instanciación de la configuración directamente en el *kernel* en caliente. La interfaz VICI pertenece a *StrongSwan* (implementación de

IPsec que permite la administración de asociaciones de seguridad).

A lo largo del documento, se van a ir mostrando diferentes apartados con información fundamental para la comprensión del trabajo. En primer lugar, se iniciará al lector en las principales tecnologías que se han usado para el desarrollo del proyecto, junto a una pequeña contextualización para situar el desarrollo del proyecto, todo ello ubicado en la sección 2. Acto seguido, haremos énfasis en los objetivos del proyecto y la metodología que hemos usado en la sección 3. Continuaremos con un desarrollo relacionado con el diseño teórico de una solución al problema propuesto en esta introducción en la sección 4 para finalmente mostrar el despliegue final del escenario y las pruebas realizadas sobre este, mostrados en la sección 5. En estas pruebas, simulamos todo el comportamiento que tiene la red desde el momento que se aplican las configuraciones en cada uno de los nodos hasta que se han aplicado las diferentes configuraciones y se permite un tráfico seguro entre nodos, donde terminaremos generando tráfico ente los dispositivos y comprobando que, efectivamente, el tráfico va protegido.

2. Estado del arte

Esta sección tiene como objetivo iniciar al lector en los protocolos, interfaces y tecnologías más importantes que se usan en el desarrollo del proyecto, donde se hablará entre otras cosas de la motivación, el funcionamiento y la estructura de cada uno de ellos. En concreto, se habla sobre el paradigma SDN, el protocolo NETCONF, el protocolo IPsec, el protocolo IKE, la implementación escogida para el control de IKE y la interfaz de VICI, poniendo finalmente al lector en el contexto donde está enmarcado el desarrollo de la práctica.

2.1. SDN (*Software Defined Networking*)

2.1.1. Motivación y descripción.

El paradigma SDN (*Software Defined Networking*) [2] presenta una nueva manera de gestionar la red, empezando por la posibilidad de desacoplar los planos de datos y de control, de tal manera que una entidad central, como es el controlador, se encarga de indicar a cada dispositivo como tiene que enrutar los paquetes que recibe (plano de control), y los dispositivos simplemente se limitan a seguir las directrices que les indica la entidad central de cara al tráfico de la red (plano de datos). Una de las claves de estos cambios, se basa en el conocimiento que tiene el controlador, donde posee una visión global de la red, y puede tomar decisiones dinámicamente para ofrecer un mejor servicio general.

De esta manera, por ejemplo se podría detectar en una cierta parte de la red una alta congestión, y podría indicarse a ciertos dispositivos que encaminen el tráfico por otro camino donde haya menos congestión. Con esta jerarquía centralizada, se obtiene un rendimiento mucho más alto en el funcionamiento de la red global, obteniendo entre otras cosas una alta calidad de servicio (QoS) y una mayor eficiencia y control general a la hora de explotar el uso de la red, que son actualmente algunas de las prioridades, sobre todo para las grandes empresas que ofertan diferentes servicios basados en la red. Destacamos la importancia de estas empresas, ya que son a su vez estas las que invierten en la investigación y el desarrollo de nuevas técnicas y mejoras, como el caso del paradigma que estamos comentando.

2.1.2. Arquitectura

En primer lugar, es necesario destacar que la arquitectura SDN posee tres planos e interfaces muy diferenciadas, que son fundamentales para comprender su funcionamiento, y que son objetivo de comprensión en esta sección. Para ello, vamos a ir comentando cada elemento, indicando la función que adquiere dentro de la topología y la ubicación en ella. Podemos observar en la Figura 1, un diagrama que nos muestra la arquitectura general, con los diferentes componentes que vamos a ir comentando. En primer lugar, vamos a hablar acerca de los tres planos diferenciados,

que son los planos de aplicación, control y datos:

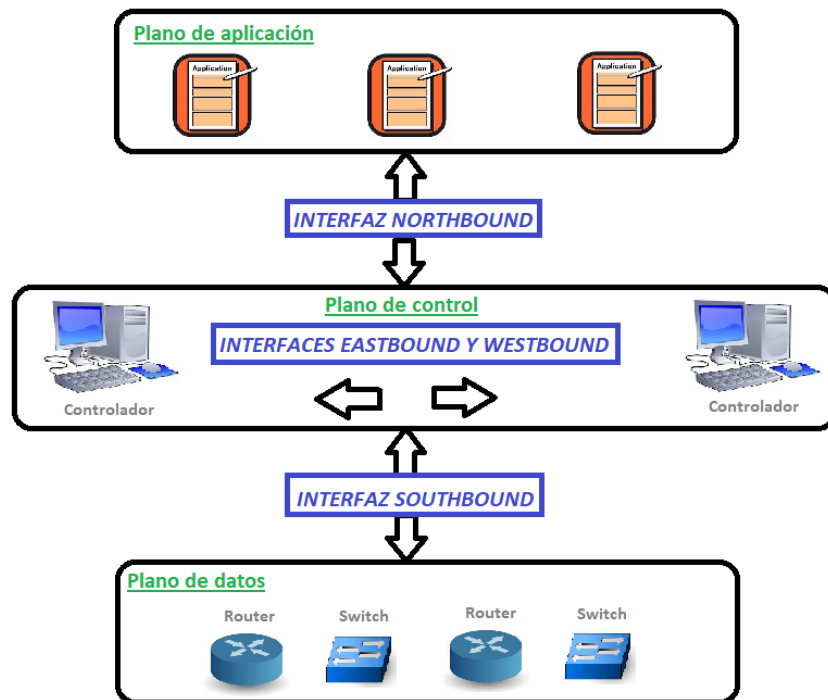


Figura 1: Arquitectura general de SDN.

- **Plano de aplicación:** Se encuentra en la parte superior de la topología, y en ella se encuentran aplicaciones que se comunican con los controladores (situados en el plano de control). Estas aplicaciones tienen como objetivo abstraer al administrador de la topología y su complejidad, de tal manera que pueda gestionar de una manera más simple y eficiente los dispositivos de la topología. Aunque esta capa no es objeto de investigación en este proyecto, es fundamental entender el lugar que ocupa en la arquitectura global del paradigma.
- **Plano de control:** En este plano está ubicada en la parte central de la topología, y es el lugar en el que se encuentran los diferentes controladores de la arquitectura. Cada uno de estos puede comunicarse directamente con cualquier elemento de la topología, empezando por la comunicación con el resto de controladores, los cuales están todos en este plano. También puede comunicarse con las aplicaciones que están en el plano superior (plano de aplicación) y con cada elemento final de la red, situados en el plano inferior (plano de datos). El controlador es la pieza fundamental de la topología, encargada de indicar a cada elemento del plano inferior la manera de proceder ante la entra-

da de diferentes paquetes, instanciando asociaciones de seguridad y políticas de enrutamiento.

- **Plano de datos:** Finalmente, tenemos localizado en la parte baja de la arquitectura este plano. En él, se encuentran los dispositivos finales, como *routers* o *switches*, encargados de encaminar el tráfico en la red. Estos dispositivos, seguirán siempre las instrucciones que les indique el controlador, que está situado en la capa superior y posee una vista global de la red, facilitando el correcto y eficiente funcionamiento de esta. Por lo tanto, los elementos de este plano se limitan únicamente a dirigir el tráfico, sin tomar ninguna decisión propia.

Una vez hemos visto los diferentes planos que posee la topología, es importantes comprender la manera en que se comunican los diferentes elementos de cada plano con el resto. Para ello, hacen uso de tres interfaces fundamentales, llamadas *Northbound*, *Eastbound* & *Westbound* y *Southbound*, que vamos a ver con detalle a continuación (reciben el nombre por los puntos cardinales que tendría en la visión global, desde la posición del controlador, puesto que se considera el cerebro de la arquitectura):

- **Interfaz *Northbound*:** Tiene como objetivo la comunicación entre el plano de aplicación y el plano de control, es decir, entre las aplicaciones y los controladores. Sirve como punto de partida para la gestión de la red, donde las aplicaciones pueden requerir en cualquier momento datos o almacenamiento, y la red puede comunicarse indicando si ha realizado con éxito la petición, o indicando el estado actual de la red. En el desarrollo de esta investigación no hacemos uso realmente de esta interfaz, puesto que nosotros indicaremos directamente desde el controlador las políticas a aplicar manualmente. Sin embargo, el papel que ocupa en la red global del paradigma nos ha llevado a expresarla en esta sección.
- **Interfaces *Eastbound* & *Westbound*:** Estas interfaces están destinadas a la comunicación entre diferentes dispositivos controladores, siendo por tanto una interfaz que se encuentra únicamente en el plano de control, sin comunicación con el resto de planos. Al igual que la interfaz anterior, esta interfaz escapa al escenario tratado en este trabajo, puesto que solo dispondremos de un controlador y no habrá comunicación con ningún otro controlador.
- **Interfaz *Southbound*:** Por último, esta interfaz se usa para simplificar la comunicación entre el plano de control y el plano de datos, es decir, entre controladores y elementos finales (como *routers* o *switches*). En esta interfaz, el controlador indicará a los diferentes dispositivos como deben distribuir y proteger el tráfico que pase por ellos, lo que permite a los elementos sincronización en función de las directrices que mande el controlador o los controladores. Principalmente, hay dos protocolos desarrollados en esta interfaz, *OpenFlow* y *NETCONF*, siendo este último el que usaremos y que explicamos en la sección 2.2.

2.1.3. Funcionamiento y caso de uso

Una vez mostrada toda la arquitectura interna del paradigma, es oportuno ejemplificar el comportamiento que tendría en un escenario real, de cara a dar al lector una visión que le ayude a comprender el funcionamiento del paradigma. Para ello, hemos diseñado un escenario simple sobre el que vamos a ir indicando paso por paso el funcionamiento de este. Podemos ver en la Figura 2 un diagrama que muestra el caso de ejemplo que vamos a usar, donde llega un paquete nuevo al *switch*.

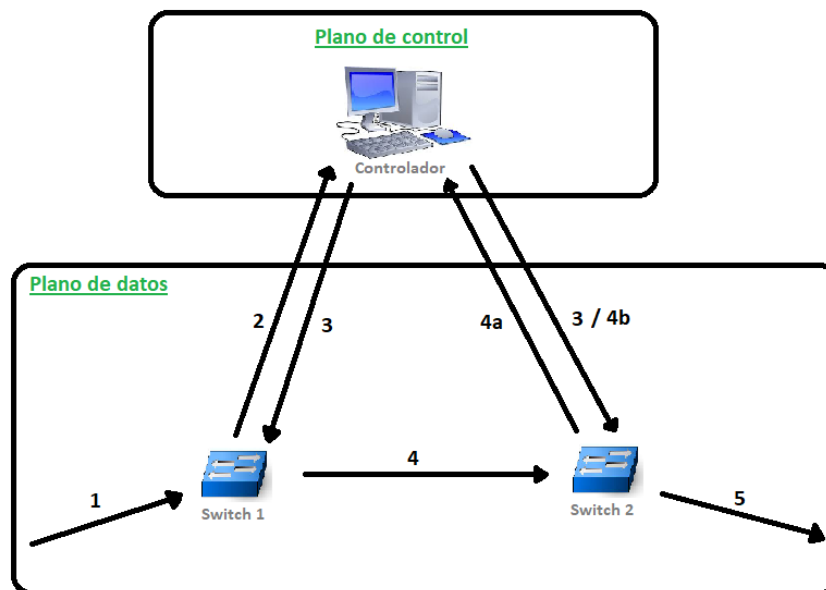


Figura 2: Funcionamiento de SDN.

- Paso 1. En este primer paso tenemos el punto de partida del ejemplo, donde llega un nuevo paquete al *switch* 1, pero este no sabe que política debe aplicar. Como tenemos los planos de control y de datos separados, debemos preguntar al controlador, el cual maneja el plano de control y nos dirá que hacer con este nuevo paquete entrante, tal y como se muestra en el paso 2. En el caso de que el *switch* 1 tuviera una política ya definida para ese paquete, lo enviaría directamente al *switch* correspondiente, saltando al paso 4.
- Paso 2. En este momento, el *switch* ha determinado que no posee una política para el paquete entrante, procediendo mediante la interfaz *Southbound* a comunicarse con el controlador, para que sea este el que le comunique que política debe aplicar. Una vez el controlador decida que ruta debe seguir el paquete concreto, instanciará la política en el *switch* que le preguntó y en el resto del camino.
- Paso 3. Una vez el controlador ha tomado las decisiones, indica tanto al *switch*

que preguntó como al resto del camino que seguirá el paquete, el camino que este debe seguir. Aquí podemos observar una de las mayores ventajas de este paradigma, puesto que al tener una visión global de la topología y un control total de esta, puede instanciar todo el camino a los componentes que lo forman, permitiendo así que los dispositivos ubicados en los siguientes pasos del camino sepan que hacer con el paquete y no tengan que ir preguntando uno a uno.

- Paso 4. Una vez ha recibido el *switch* la información necesaria para dirigir el paquete, o en el caso de que la tuviera desde un principio, aplica la política correspondiente al paquete y lo envía al siguiente punto del camino.
- Pasos 4a y 4b. Han sido añadidos en el diagrama estos dos pasos extra con la idea de dejar en evidencia la mejora que supone el uso de este paradigma. En el caso de que la figura del controlador no tuviera una visión global de la red, no podría ir aplicando la configuración a todo el camino que va a seguir el paquete, y cada elemento tendría que preguntar al controlador que hacer con el paquete que recibe, lo que haría a la red funcionar con una excesiva latencia.
- Paso 5. Cuando el *switch* 2 recibe el paquete, ya sabe que debe hacer con el paquete, puesto que el controlador le ha indicado previamente la política a aplicar. En este punto, simplemente aplica la política correspondiente y el paquete continua el camino.

2.2. NETCONF (*Network Configuration Protocol*)

2.2.1. Motivación y descripción

Ante la necesidad de definir un protocolo que intentara englobar la administración de la red, apareció el protocolo SNMP (*Simple Network Management Protocol*) [3], y se convirtió en ese momento en el protocolo de administración de red más reconocido y usado. El problema surgió cuando este protocolo no se usaba realmente para la administración de la red, sino para el control y monitoreo de esta. Detectado este problema, volvió la necesidad de disponer de un protocolo que ayudara en la tarea de la administración de la red, momento en el que apareció NETCONF (*Network Configuration Protocol*) [4], versión mejorada y ampliamente aceptada que usaremos en el desarrollo de este proyecto.

El protocolo NETCONF está diseñado para permitir al administrador de la red la manipulación de la configuración de los distintos dispositivos que conforman la red, permitiendo la instanciación, eliminación y manipulación de estas configuraciones. Todas estas operaciones son realizadas mediante RPC (*Remote Procedure Call*) [5], usado para la ejecución de código en otra máquina sin la problemática de la comunicación entre estas. Para representación de los mensajes intercambiados se hace uso de la codificación XML (*Extensible Markup Language*) [6].

Por otra parte, para aplicar la configuración de los diferentes mensajes en formato XML debe haber una validación frente a modelos YANG [7]. Estos modelos aportan la posibilidad de poder verificar la sintaxis y los campos dentro de los archivos XML, de tal manera que ayuda a limitar la posibilidad de errores en las configuraciones instanciadas, estructurados en forma de árbol. Por último, este tráfico de mensajes se realiza sobre protocolos seguros de transporte, como son SSH (*Secure Shell*) [8] o TLS (*Transport Layer Security*) [9].

2.2.2. Arquitectura

Las comunicaciones en NETCONF se realizan mediante RPC, entre cliente y servidor de la arquitectura. En este escenario, el cliente se entiende por la entidad que se encarga de gestionar la red, mientras el servidor se refiere a cada dispositivo que se configura. El protocolo está organizado en cuatro capas diferenciadas, las cuales podemos observar claramente en la Figura 3, donde observamos la capa correspondiente y a su lado ejemplos de información o protocolo que maneja cada una de ellas, las cuales vamos a ir comentando en detalle a continuación.

- **Contenido:** Esta capa es la que permite manipular los distintos datos de configuración de los distintos dispositivos de la red.
- **Operaciones:** Define el conjunto de operaciones que se invocan como métodos de RPC, codificados en XML, los cuales se ven con más detalle en la sección 2.2.3.

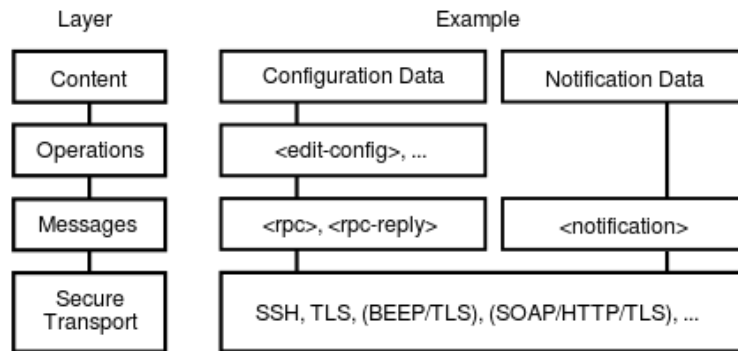


Figura 3: Arquitectura general de NETCONF [10].

- **Mensajes:** Esta capa nos proporciona un mecanismo simple e independiente de la capa de transporte para codificar RPC y notificaciones.
- **Transporte:** Esta capa nos proporciona una ruta de comunicación entre servidor y cliente. NETCONF puede superponerse en cualquier protocolo de transporte que posea unos requisitos específicos. Estos requisitos se resumen en que el protocolo esté orientado a conexión, que soporte SSH y que cumpla con los requisitos de seguridad de NETCONF (autenticación, integridad, confidencialidad y protección contra reenvío), ya que este delega el establecimiento de conexiones seguras en la capa de transporte.

También define NETCONF diferentes modelos de datos o *database* con la finalidad de poder ejecutar operaciones de configuración sobre ellos, donde no se añaden datos ni comandos. Estos modelos de datos son un conjunto de operaciones usadas para hacer que el dispositivo que las aplique tenga el estado deseado. Se definen tres modelos de datos, los cuales son *running*, *candidate* y *startup*, tal y como comentamos a continuación:

- **Database running:** Este modelo de datos contiene en todo momento toda la configuración actual del dispositivo. Solo está presente una vez en el dispositivo y está activo siempre que el dispositivo esté funcionando, de tal manera que no estará disponible la información cuando el elemento esté apagado.
- **Database candidate:** Modelo de datos que nos sirve para disponer en el dispositivo una configuración que pueda ser modificada sin afectar a la configuración actual en ese momento, pudiendo ser instanciada como configuración actual en cualquier momento. Para instanciar en cualquier momento la configuración que posea este modelo, debemos usar la operación *commit*. Al realizar la operación, la configuración que tenemos en *candidate* pasa a ser también la configuración actual (*running*). Por contra, podemos realizar el proceso contrario usando *discard-changes*, operación que nos copia la configuración actual

dentro de *candidate*, eliminando así los cambios que se hubieran realizado en ella. Por último indicar que la configuración de este modelo se puede compartir entre varias sesiones, por lo que se debe asumir que otras sesiones pueden modificar la configuración candidata al mismo tiempo, siendo aconsejable que un cliente bloquee la configuración candidata antes de modificarla.

- *Database startup*: Este modelo de datos contiene la configuración que se debe cargar cada vez que se inicia el dispositivo, es decir, el dispositivo al arrancar carga esta configuración, pasando esta a ser también la configuración actual *running* en ese momento. Tenemos disponible una operación, *copy-config*, que nos permite copiar en cualquier momento la configuración actual a este modelo.

2.2.3. Operaciones

Dentro de las capas que comentábamos anteriormente, teníamos la capa de operaciones, que se invocan como métodos de RPC y son codificadas en XML. Estas operaciones tienen como objetivo el control de configuraciones y estado de los diferentes dispositivos de la red, donde disponemos de opciones para borrar, copiar, instanciar y configurar dentro de las *database* del servidor. Vamos a ir comentando el objetivo y la funcionalidad que ofrecen todas las operaciones disponibles a continuación:

- *get-config*: Tiene como objetivo devolvernos la configuración de una *database* específica, o una parte de esta. Los parámetros que nos ofrece son: *source* para indicar la *database* de la que queremos obtener la configuración y *filter* si queremos indicar una parte de la configuración concreta a obtener.
- *edit-config*: La función de esta operación es cargar una configuración concreta o una porción de esta en una *database* objetivo, donde en caso de no existir se crearía una nueva. No está definido el resultado de la operación cuando esta contiene múltiples operaciones que se aplican al mismo punto de la *database*. Los parámetros que nos ofrece son: *target* para indicar la *database* destino de la configuración y la operación (*operation*) a realizar con esta, donde los valores que se puede indicar dentro del parámetro de operación son los siguientes:
 - *merge*: La configuración ya existente en la *database* correspondiente se fusiona con la nueva configuración que contiene el elemento contenedor de este atributo. Es el comportamiento por defecto.
 - *replace*: Se reemplaza en la configuración existente cualquier configuración relacionada con la configuración a aplicar. En el caso de no encontrar los datos de configuración en la *database* destino, los crea.
 - *create*: Carga la información a aplicar en la configuración de la *database* destino, en el caso de que no exista ya. Si la información ya se encontraba aplicada, devolverá un error indicando que ya existían esos datos (*data-exists*).

- *delete*: Elimina la información que encuentre en la *database* que coincida con la configuración indicada en la operación. Si no encuentra esa información para borrarla, indicará un error (*data-missing*).
- *remove*: Funciona exactamente igual que *delete*, con la única diferencia en el caso de que la información no exista, no indicando ningún error e ignorando la operación.
- *copy-config*: Crea o sustituye una configuración completa de una *database* por otra configuración completa. Si ya existía la configuración en el objetivo, esta es reemplazada. Los parámetros que nos ofrece son: *target* para indicar la *database* destino y *source* para indicar que configuración vamos a aplicar en el destino.
- *delete-config*: Elimina la configuración de un *database*, no siendo aplicable a la configuración que se está usando en ese momento (*running*). El único parámetro que nos ofrece esta operación es la *database* a eliminar.
- *lock*: Nos permite la posibilidad de bloquear la configuración de una *database*, permitiendo que un usuario pueda aplicar cambios sin temor a que algún otro usuario o servicio modifique los datos en ese momento. Esta operación fallará si la *database* que se intenta bloquear está en ese momento bloqueada por otro usuario o servicio. El único parámetro que nos ofrece es *target*, para indicar la *database* destino del bloqueo.
- *unlock*: Usado para desbloquear una *database* que ha sido previamente bloqueada usando la operación *lock*.
- *get*: Obtiene información acerca del estado de un dispositivo y su configuración activa (*running*). Posee un parámetro *filter* para indicar una porción concreta del dispositivo a obtener, en caso de no querer obtener toda la información.
- *close-session*: Solicita una finalización correcta de la sesión de NETCONF. Cuando un dispositivo lo recibe, debe anular todos los bloqueos relacionados con la sesión e ignorar nuevas operaciones relacionadas con esa sesión.
- *kill-session*: Es un caso más estricto de la operación *close-session*, donde se fuerza la terminación de la sesión de NETCONF. Cuando un dispositivo lo recibe aborta todas las operaciones que esté realizando en ese momento relacionadas con la sesión que se va a terminar.

2.3. IPsec (*Internet Protocol Security*)

2.3.1. Motivación y descripción

Una vez tenemos comprendido el escenario sobre el que enfocamos el proyecto, es fundamental dotar a todas las conexiones y el tráfico que fluye por estas, de una robusta seguridad. Es en este punto cuando aparece en escena IPsec [11], un conjunto de protocolos que nos ofrece seguridad en las comunicaciones en la capa de IP. En la mayoría de las ocasiones, este delega en el protocolo IKE (como veremos más adelante) la negociación inicial de claves y la gestión de estas. Por lo tanto, a la hora de desarrollar la seguridad en la comunicación entre elementos de este proyecto, usaremos este conjunto de protocolos.

IPsec usa asociaciones de seguridad para formar las diferentes reglas de protección, donde se pueden indicar entre otras cosas algoritmos de cifrado, modos de cifrado y parámetros de red. Posee una importante ventaja frente a otros protocolos de securización en la red, puesto que al estar implementado en la capa IP le dota de una alta flexibilidad y le da la posibilidad de proteger protocolos de las capas inferiores. En los siguientes apartados indagaremos más en estas asociaciones.

2.3.2. Arquitectura y procesamiento

Las asociaciones de seguridad contienen la información necesaria para establecer una comunicación segura entre dos *hosts*, donde se requieren dos asociaciones de seguridad para que se comuniquen en ambas direcciones de forma segura. IPsec nos proporciona mecanismos de seguridad relacionados con la encriptación y la autenticación de paquetes, aplicándose estos a los paquetes que indica el archivo de políticas global.

Una asociación de seguridad se define principalmente por tres elementos, que son un protocolo que proporciona seguridad (que puede ser tanto AH como ESP), la dirección IP destino y un índice de parámetros de seguridad (SPI), elementos que vamos a ver en más detalle a continuación.

- AH (*Authentication Header*) [12]: Está pensado para dotar a cada paquete de autenticación e integridad. Coloca una firma digital en cada paquete, protegiendo el contenido de la interferencia de terceros ajenos a la comunicación. Si se modificara el contenido del paquete, se detectaría inmediatamente el cambio en los datos (integridad). Para conseguir esto, se calcula un HMAC (*Hash Message Authentication Code*) usando el contenido del propio paquete junto a una clave, donde los algoritmos más usados son MD5 (*Message-Digest Algorithm 5*) [13] y SHA1 (*Secure Hash Algorithm 1*) [14]. Opcionalmente, puede proteger también frente a ataques de repetición usando técnicas de ventana deslizante.

- ESP (*Encapsulating Security Payload*) [15]: El objetivo principal es dotar al usuario de confidencialidad, proporcionando también integridad y autenticidad. Por lo tanto, mientras que AH evita la posibilidad de manipular de un paquete, ESP asegura que la información paquete no pueda ser accesible y esté encriptada. ESP también permite cifrado sin autenticación, pero está totalmente desaconsejado debido a serios problemas de seguridad. En este caso, la cabecera del paquete no va protegida. Los algoritmos de cifrado que usa normalmente son 3DES (*Triple Data Encryption Standard*) [16] y AES (*Advanced Encryption Standard*) [17]. No disponer de autenticación permite vulnerabilidad a ataques de repetición, y no disponer de confidencialidad le hace igual de vulnerable que AH.
- SPI (*Security Parameters Index*): Es una identificación que se añade a la cabecera del paquete, con el objetivo de ayudar a diferenciar entre distintos flujos de tráfico que pueden poseer diferentes cifrados y algoritmos. Es necesario en una asociación de seguridad puesto que permite al sistema seleccionar que política aplicar al paquete entrante. Está presente tanto en ESP como en AH.

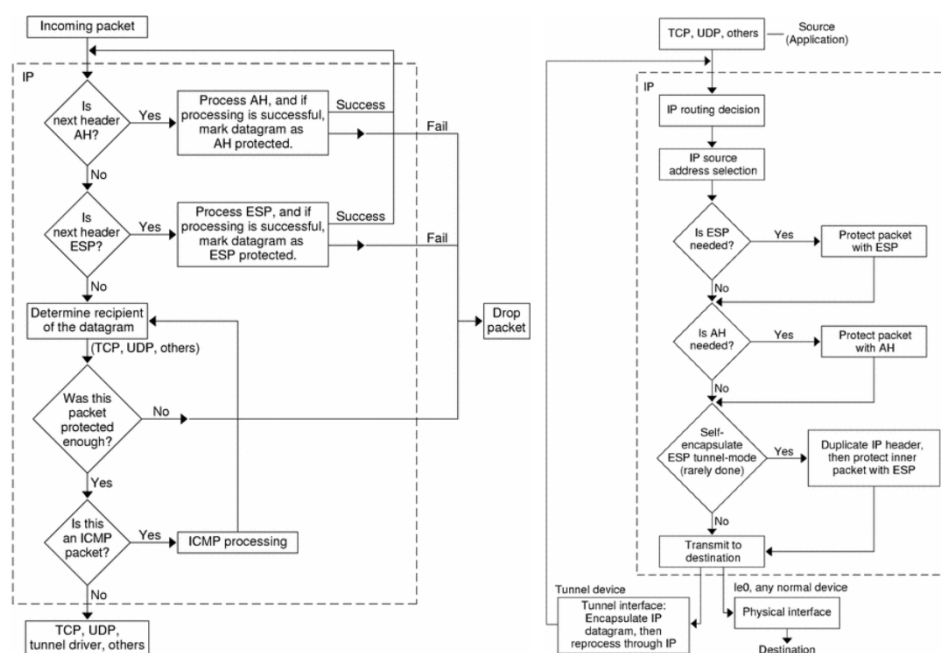


Figura 4: Procedimiento al invocar a IPsec, flujo entrante y saliente [19].

Para comprender su funcionamiento, podemos observar en la Figura 4, el procedimiento que se hace al paquete cuando es enviado por el usuario y se invoca a IPsec. Empieza comprobando si el paquete requiere AH, y posteriormente si es ESP, donde en ambos casos si se procesa correctamente se marca el paquete como protegido con el método correspondiente y se empieza el proceso de nuevo, haciendo las

comprobaciones. Importante destacar que AH y ESP no pueden estar en la misma política a aplicar, de tal manera que si se requiere uso de los dos, se deben tener dos reglas separadas. Se pueden aplicar múltiples reglas, puesto que se vuelve al inicio del proceso cuando se aplican políticas relacionadas con AH o ESP indistintamente. Por contra, en caso de fallar en cualquiera de los métodos el paquete se descarta. Acto seguido se comprueba el contenido del paquete y si esta correctamente protegido, donde si no está bien protegido se descarta y en otro caso se procesa. Si el paquete no es de tipo ICMP, se manda al destino, ya sea por modo túnel o modo transporte.

En el caso de que el usuario reciba un paquete IPsec, se debe realizar el proceso contrario, como se puede observar en la segunda parte de la figura. En este momento, se comprueba si la protección del paquete requiere AH o ESP y se procesa en función de los requerimientos. Una importante diferencia con el flujo de entrada, es que ese comprueba en último lugar la posibilidad de tener un paquete duplicado (protegiéndose así de ataques de repetición), y acto seguido se procesa el paquete.

IPsec también nos define tres bases de datos para asegurarse un correcto procesamiento del tráfico IP, comprobando tanto la información de autenticación como de cifrado. Las entidades son SAD, SPD y PAD [21], las cuales se describen a continuación.

- **SPD** (*Security Policy Database*): Esta base de datos nos especifica el tráfico que debe ser protegido mediante IPsec y el tráfico que debe dejar pasar. Esta base de datos es consultada para todo el tráfico entrante y saliente. Se usará la primera política que coincida con los datos del paquete, procesándolo en base a las reglas que contenga. En el caso de no encontrar ninguna regla, el paquete será directamente descartado.
- **SAD** (*Security Association Database*): Por su parte, esta base de datos contiene información relacionada con el procesamiento de IPsec, de tal manera que se comprobará en la entrada correspondiente que protección debe aplicarse al archivo. A esta base de datos se accede únicamente cuando la regla en SPD relacionada con el paquete nos indica que precisa de procesamiento IPsec.
- **PAD** (*Peer Authorization Database*): Finalmente, esta base de datos nos proporciona un enlace entre el protocolo IKE e IPsec. En este punto se negocian los parámetros necesarios para la seguridad en la comunicación con el destino, con fines de autenticación.

Podemos observar en la Figura 5 las comprobaciones que se hacen en un paquete protegido con IPsec en el flujo de salida y en la Figura 6 podemos observar el flujo de entrada, sobre las que vamos a apoyar la explicación del funcionamiento general. Cuando un nuevo paquete llega, lo primero que se hace es comprobar en la SPD si existe una regla asociada a ese paquete (se selecciona la primera que coincida con

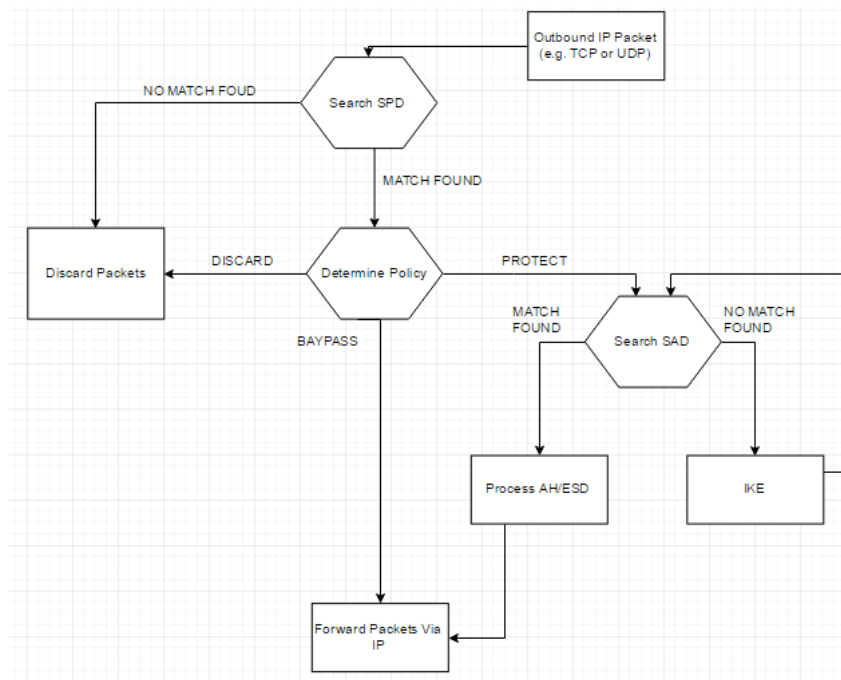


Figura 5: Administración de IPsec, flujo saliente [20].

el paquete), donde en caso de no existir ninguna regla es directamente descartado. Si se ha encontrado una regla, se pueden realizar tres acciones con el paquete, las cuales son:

- Descartar: Ocurrirá en el caso de que la política indique explícitamente que ese paquete no debe continuar, en cuyo caso se elimina el paquete y no sale a la red.
- Proteger: Sucederá cuando en la regla se indique que el paquete debe ser protegido mediante IPsec, donde se accederá a la SAD para ver que políticas hay que aplicarle en concreto.
- Dejarlo pasar: Solamente se le dejará pasar si en la política del paquete se indica que no necesita protección pero se puede enviar, de tal manera que sale a la red hacia el destino.

En el caso de que la política a aplicar sea proteger el paquete, se accede a la SAD y esta comprueba si dispone de reglas a aplicar para el paquete. En caso afirmativo, se procesan las reglas en el paquete y se manda al destino. En caso negativo, se comunica con IKE para gestionar una nueva asociación entre el origen y el destino, y esta regla se guarda en la SAD. Por lo tanto, cuando se vuelve a consultar, ya se dispone de una regla concreta, la cual se aplica, y finalmente se manda el paquete.

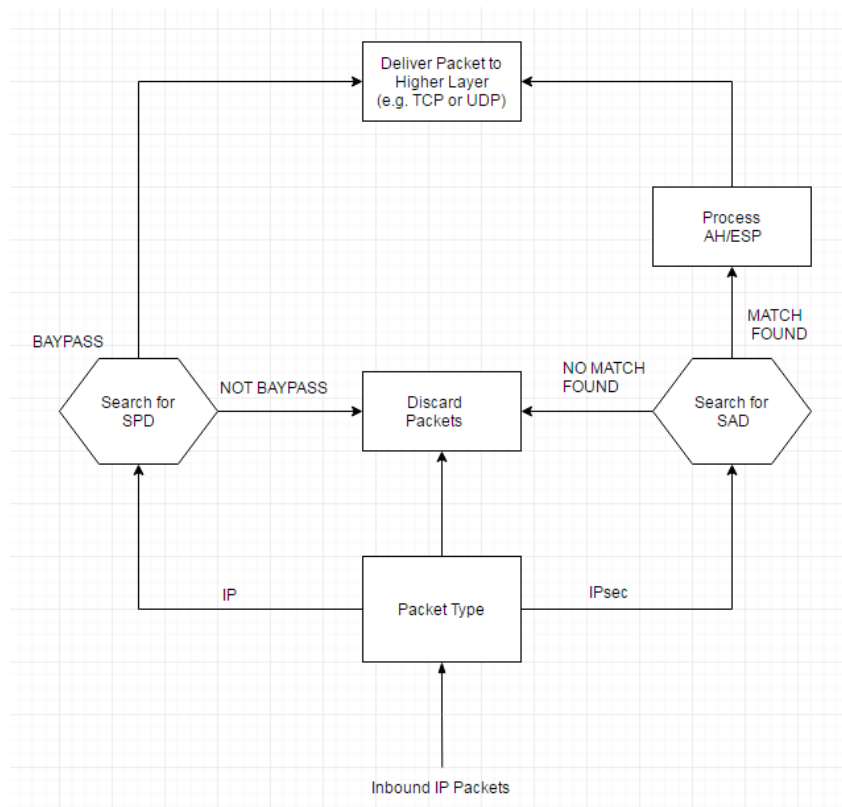


Figura 6: Administración de IPsec, flujo entrante [20].

Por otra parte, cuando se recibe un paquete, se comprueba en primer lugar si el paquete viene securizado (IPsec) o no (IP). En caso negativo, se comprueba directamente la existencia de reglas frente a la SPD, decidiendo si se deja pasar al paquete o se descarta. En caso afirmativo, se comprueba frente a la SAD, y en caso de encontrar una regla, la aplica y deja pasar el paquete. En caso de no encontrar una regla para ese paquete es directamente descartado.

2.3.3. Modos de funcionamiento

IPsec posee dos modos fundamentales de funcionamiento, los cuales son modo transporte y modo túnel. Ambos son compatibles tanto con AH como con ESP, y vamos a ir comentando las ventajas e inconvenientes de cada uno, junto a su funcionamiento.

El modo túnel es el modo predeterminado de trabajo de IPsec. Este modo protege la totalidad del paquete, donde en primer lugar procede con la encriptación del paquete original y luego le añade una cabecera con la dirección IP del dispositivo ubicado al otro lado del túnel, enviándolo finalmente al dispositivo destino.

Este modo está pensado para usarse entre dispositivos de la red que no sean finales, donde un ejemplo muy común suele ser entre dos *routers* que dan salida a internet a dos organizaciones diferentes, suponiéndose la confianza en el interior de estas redes, como se puede observar en la Figura 7.

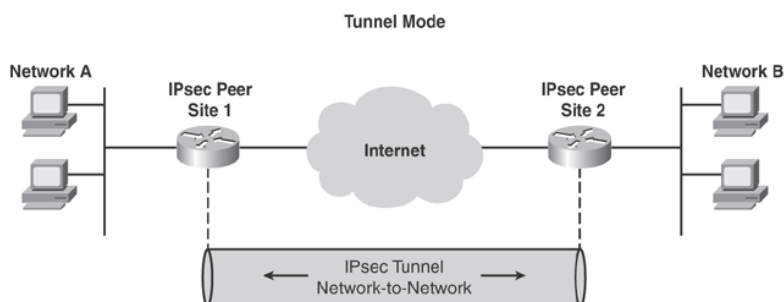


Figura 7: Funcionamiento de modo túnel [22].

Por su parte, el modo transporte nos proporciona protección únicamente a las capas superiores, es decir, la carga útil del paquete. Esta carga útil protegida se encapsula, quedando insertada entre encabezados y campos finales de IPsec. Este modo ha sido pensado para la comunicación directa entre dos usuarios finales o *hosts* de la red, de tal manera que el paquete hace todo el recorrido protegido, tal y como puede verse en el ejemplo de la Figura 8.

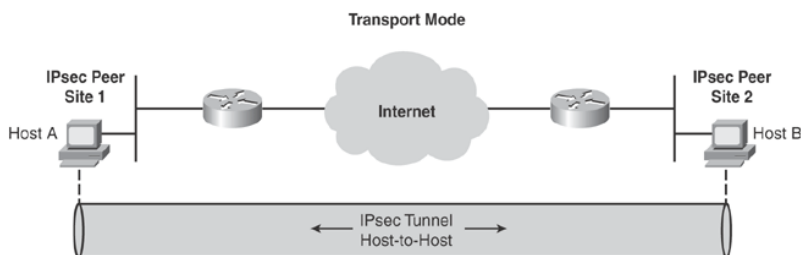


Figura 8: Funcionamiento de modo transporte [22].

Tal y como hemos comentado, tanto AH como ESP pueden funcionar en modo túnel y en modo transporte. Para el caso de AH, podemos ver en la Figura 9 el formato que tiene el paquete en ambos modos. Cuando actúa en modo túnel autentica el paquete en su totalidad, al que le añade una cabecera de autenticación y la nueva cabecera (indicando el *peer* con el que estás creando el túnel). En el caso del modo transporte, la diferencia radica en que la nueva cabecera que se crea y se usa para encapsular el paquete completo, pasa a ser la que contenía el paquete original, cambiando únicamente unos pequeños valores. En este segundo caso puede verse como realmente el modo transporte no protege todo el paquete, sino solo la

carga útil, dejando al descubierto todo el contenido de la cabecera.

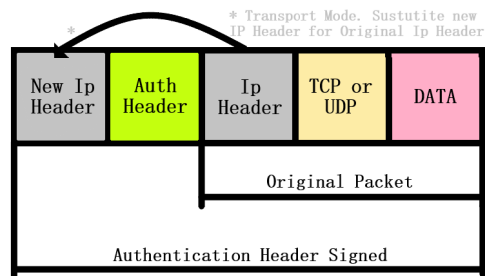


Figura 9: Cabeceras AH, modo túnel vs modo transporte.

En el caso de ESP, para el modo túnel podemos observar como se cifra y autentica todo el paquete. Para ello, se encapsula el paquete original y pasa a ser la carga útil de otro nuevo paquete, donde añadimos la nueva cabecera. A diferencia de AH, en este caso encapsulamos el paquete inicial dentro tanto de una cabecera como de un tráiler ESP y un tráiler de autenticación ESP. Una vez el paquete está formado, se le añade una nueva cabecera con el *peer* con el que formamos el túnel, y finalmente se envía el paquete. En modo transporte, la diferencia reside (al igual que en AH) en la nueva cabecera IP, que realmente es la misma que contenía el paquete original, cambiando únicamente unos pocos valores. Podemos observar la estructura del paquete en ambos modos en la Figura 10.

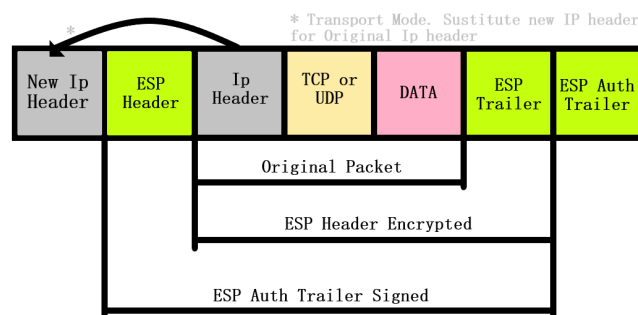


Figura 10: Cabeceras ESP, modo túnel vs modo transporte.

2.4. IKE (*Internet Key Exchange*)

2.4.1. Motivación y descripción

Tal y como hemos ido comentando previamente en la descripción de IPsec, tenemos un protocolo encargado de la negociación de los parámetros de una asociación de seguridad dentro de IPsec, siendo este IKE (*Internet Key Exchange*) [23]. Trabaja con intercambio seguro de claves *Diffie-Hellman* [24] para obtener el secreto compartido de cada sesión, usando normalmente sistemas de clave pública. El uso de esta manera de instanciar las asociaciones de seguridad supera en eficiencia y comodidad a la manera tradicional, que se basaba en ir introduciendo manualmente la información directamente en las distintas SAD.

Puesto que la primera versión de IKE poseía una alta complejidad y un exceso innecesario de mensajes en la negociación inicial, se mejoró el protocolo en una segunda versión, IKEv2 [25]. Esta mejora de diseño consiguió simplificar la negociación inicial, de tal manera que se necesitan intercambiar cuatro mensajes (en puesto de ocho). También incorporó soporte para movilidad (*mobike*) y una mejora en los mecanismos de criptografía usados, al simplificar estos usando mecanismos similares al protocolo ESP. Finalmente, dentro de las mejoras principales que introdujo, cabe destacar la correcta respuesta frente a ataques de DoS (*Denied of Service*), solventada gracias a no realizar el procesamiento hasta que no se asegura de la existencia del extremo que ha realizado la petición. Cuando la negociación finaliza, ambos dispositivos poseen una clave compartida previamente (*pre-shared key*) con la que proteger la comunicación.

Por lo tanto, a modo de resumen, IPsec se apoya en IKEv2 para la negociación inicial de claves, ya que IPsec por sí solo no posee esta característica. IKEv2 trabaja sobre el nivel de aplicación usando UDP, y una vez se han acordado las claves iniciales, éstas se pasan a la pila IPsec, que se está ejecutando en el propio *kernel*. IKE también se usa para la renovación de las claves negociadas previamente, una vez estas expiren, volviendo a realizar la negociación.

2.4.2. Arquitectura y funcionamiento

Como hemos comentado antes, la realización de la negociación se realiza solo con cuatro mensajes. En el intercambio, tenemos dos entidades involucradas, conocidas en IKE como *initiator* (el dispositivo que inicia el intercambio) y *responder*. Para establecer una asociación de seguridad, previamente se necesita unos intercambios iniciales, como podemos ver en el diagrama de la Figura 11, que vamos a ir comentando en detalle.

En primer lugar tenemos el intercambio *IKE.SA_INIT*, que consta de un par de mensajes, y tiene como objetivo la negociación de algoritmos criptográficos y la

realización de un intercambio *Diffie-Hellman*. El primer mensaje es mandado por la entidad que inicia el intercambio (*initiator*), y el mensaje contiene los siguientes elementos: una cabecera *HDR* que contiene los distintos parámetros de seguridad (una o varias SPI), versiones y distintos *flags*. También indica en *SAi1* los algoritmos criptográficos que el *initiator* admite para la asociación de seguridad de IKE. Finalmente, indica el valor inicial de *Diffie-Hellman* en *KEi1* y el valor inicial de *nonce* en *Ni*.

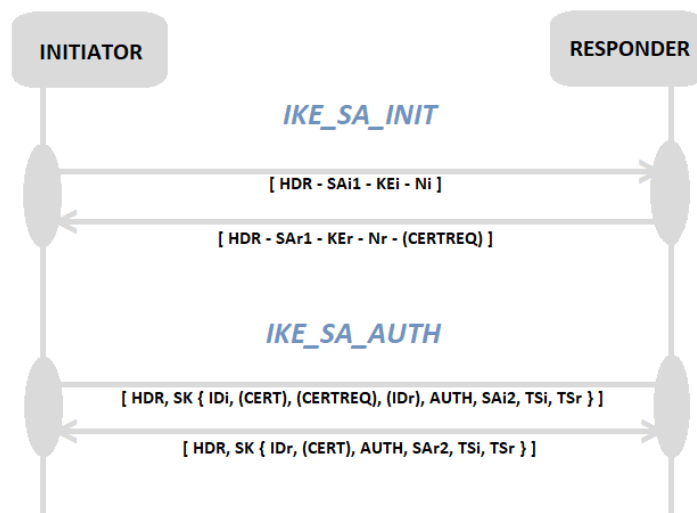


Figura 11: Negociación inicial de IKE.

Una vez ha recibido el *responder* el mensaje mencionado, responde con un mensaje similar. Selecciona de entre los algoritmos criptográficos que ofrece la otra entidad los que considere necesarios y los envía en *SAR1*, después termina el intercambio *Diffie-Hellman* y lo intercambia con *KEr*, y finalmente envía su correspondiente *nonce* en el campo *Nr*. Adicionalmente, este mensaje pide mediante el uso de *CERTREQ* un certificado para comprobar la veracidad de la identidad de la otra entidad involucrada, en este escenario sería *initiator*.

En este punto de la negociación inicial, ambas entidades pueden generar una semilla llamada *KEYSEED*, a partir de la cual derivarán diferentes claves que usarán para el cifrado y la protección de la integridad. Todos los mensajes intercambiados a partir de este punto están protegidos completamente, a excepción de las cabeceras. Acto seguido tenemos el intercambio *IKE_SA_AUTH*, que consta también de un par de mensajes, teniendo como objetivo autenticar los mensajes del paso previo, crear una primera asociación de seguridad e intercambiar identidades y certificados. En el primer mensaje que se manda podemos observar como se envía la cabecera en claro, pero el resto del contenido está protegido por las claves previamente calculadas, de

las cuales solo tienen conocimiento las dos entidades implicadas. Dentro del contenido del mensaje, se manda la identidad en el campo *IDi* y con integridad se protege el contenido del mensaje usando la información del campo *AUTH*.

Opcionalmente, también puede enviar su certificado o una lista de sus certificados mediante *CERT* y otro certificado o lista de certificados en los que confía mediante *CERTREQ*. En caso de incluir algún certificado en *CERT*, el primero de estos debe contener la información necesaria para verificar el campo *AUTH*. También es opcional indicar la identidad con la que se va a comunicar, indicada en el campo *IDr*, útil cuando se tienen varias identidades tras una misma dirección. Se indica también en el campo *SAi2* los parámetros para iniciar la creación de una asociación de seguridad. Por último, se indican parámetros usados como selectores de tráfico, en los campos *TSi* y *TSr*.

Una vez ha recibido y procesado el mensaje el *responder*, envía un mensaje en respuesta bastante similar al que ha recibido, donde manda su identidad en *IDr*, una lista opcional de certificados con *CERT*, protege la integridad del mensaje con *AUTH* y completa la negociación de la asociación de seguridad con *SAr2*. Por último, contiene también parámetros usados como selectores de tráfico, en los campos *TSi* y *TSr*. En estos intercambios, ambas entidades deben asegurarse de que todas las firmas y claves se han calculado correctamente.

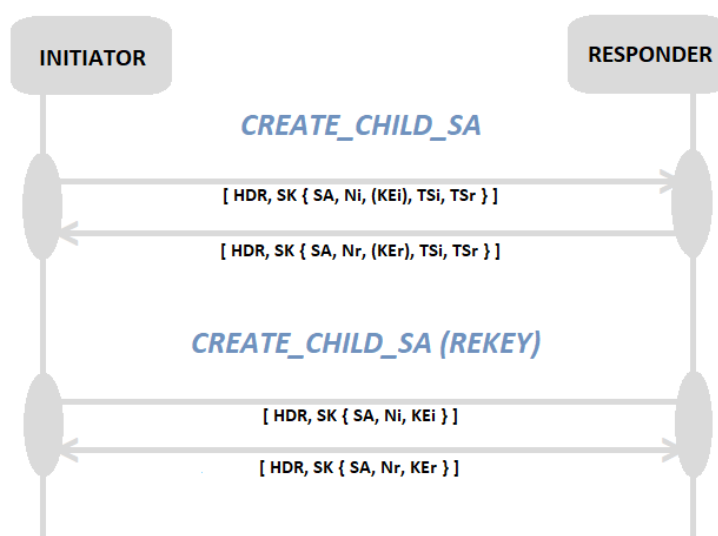


Figura 12: Negociación de una asociación de seguridad de IKE.

Partiendo de la negociación inicial descrita anteriormente ya realizada, podemos querer crear una nueva asociación de seguridad o volver a cargar una asociación de seguridad que ha caducado. Observamos los intercambios que se realizan en la

Figura 12, donde en ambos casos son un par de mensajes que se intercambian las dos entidades participantes, *initiator* y *responder*.

En el caso de querer crear una nueva asociación de seguridad, se realiza el intercambio *CREATE_CHILD_SA*, donde se puede observar que los pasos iniciales ya se han realizado, puesto que tenemos toda la información protegida a excepción de la cabecera. El *initiator* envía un mensaje en el que indica la siguiente información: envía una posible asociación de seguridad en el parámetro *SA*, un valor *nonce* en *Ni* y contiene también parámetros usados como selectores de tráfico, en los campos *TSi* y *TSr*. Opcionalmente, se puede añadir en *KEr* un valor de *Diffie-Hellman*.

Por parte del *responder*, cuando ha procesado el mensaje entrante, responde con unos parámetros similares a los recibidos. La información que contiene el mensaje de respuesta es: el parámetro *SA* con la oferta aceptada con respecto a la asociación de seguridad, el valor *nonce* correspondiente en *Nr*, los selectores de tráfico en *TSi* y *TSr*, y finalmente se indicará en el campo *KEr* el valor de *Diffie-Hellman*, solo en el caso de que el mensaje anterior también indicara información al respecto.

Por último, podemos querer volver a instanciar una asociación de seguridad que ha caducado. Para ello volvemos a usar *CREATE_CHILD_SA*, indicando esta vez diferentes parámetros. Cuando se vuelve a instanciar una asociación, realmente se borra la que ha caducado y se introduce una nueva. La diferencia con el caso anterior radica en el campo de la asociación de seguridad *SA*, en el cual se debe indicar un nuevo valor de SPI, y en la ausencia de los campos relacionados con los selectores de tráfico.

2.5. Implementaciones *OpenSource* de IKE

A la hora de hacer uso de IKE en nuestro trabajo, debemos tener claro que implementación vamos a usar de entre todas las disponibles. Principalmente, podemos distinguir tres implementaciones, que son *OpenSwan*, *LibreSwan* y *StrongSwan*.

En primer lugar, *OpenSwan* [26] se define como el software de red privada por defecto para la comunidad de Linux. Es una implementación de IPsec que tiene soporte para la mayoría de las extensiones relacionadas con IPsec, donde, principalmente, incluye el uso de IKEv2, el uso de certificados X509 y el uso de NAT transversal. Unos años después de la aparición de *OpenSwan*, apareció una variante de este proyecto que fue llamada *LibreSwan*.

Cuando apareció la implementación de *LibreSwan* [27], se popularizó rápidamente y sustituyó casi por completo el uso de la implementación previa. *LibreSwan* es una implementación de software libre del protocolo VPN estandarizada que utiliza IPsec e IKE, siendo compatible tanto con la versión 1 como la versión 2 de IKE. Dentro de Linux, utiliza la pila IPsec *xfrm* y la biblioteca criptográfica NSS. La implementación de las instrucciones en *LibreSwan* es totalmente compatible con *OpenSwan*.

Por último lugar, tenemos *StrongSwan* [28], siendo una implementación de IPsec multiplataforma y de código abierto que nos permite la administración de las distintas asociaciones de seguridad en los sistemas Linux inicialmente, aunque con el paso del tiempo se ha adaptado a otra gran variedad de plataformas. Se centra principalmente en la autenticación usando certificados de clave pública X509 y almacenamiento de claves. Posee también un demonio para la implementación de IKE (en concreto IKEv2) llamado *charon*, estableciendo asociaciones de seguridad entre diferentes dispositivos (a partir de la versión 5 del demonio, es compatible con IKEv1).

Principalmente, *StrongSwan* se centra en ofrecer una configuración simple, potentes métodos tanto de encriptación como de autenticación y políticas robustas de IPsec para redes virtuales complejas. A todo ello, se le añade una interfaz llamada VICI (*Versatile IKE Configuration Interface*), que nos permite la posibilidad de cargar en caliente la configuración recibida, omitiendo el uso de ficheros de configuración o el reinicio de servicios. Esta última ha sido la implementación elegida para el desarrollo del trabajo, debido a la interfaz VICI, y en el despliegue de nuestro escenario, realizará las negociaciones pertinentes de asociaciones de seguridad, y nos permitirá la implantación en caliente comunicándonos directamente con el *kernel* (haciendo uso de VICI, el cual exponemos en el siguiente apartado). Nos ofrece también diferentes comandos, como *swanctl*, para la configuración del dispositivo.

2.6. VICI (*Versatile IKE Configuration Interface*)

2.6.1. Motivación y descripción

Los paquetes software que implementan IKE carecen de posibilidades de controlar y monitorear el demonio que lo ejecuta (*charon*) [29], limitándose únicamente a la configuración. El problema está presente principalmente porque las interfaces existentes de arranque no se han diseñado nunca para automatizarse, debido a que la creación de las secuencias de comandos para estas herramientas es complejo, y manejar este tipo de información resulta una tarea ardua.

En el desarrollo del proyecto usamos *StrongSwan*, usándose a menudo para proporcionar la funcionalidad del servicio IKE en un sistema a medida para necesidades específicas. En muchas ocasiones los desarrolladores de estos sistemas tienen la necesidad de automatizar la configuración y el control del demonio IKE, requiriendo interactuar con la infraestructura de *StrongSwan* sin intervención humana. Para intentar paliar este problema se hace uso de esta interfaz llamada VICI (*Versatile IKE Configuration Interface*) [30], que fue diseñada para permitir la gestión del demonio de IKE desde un módulo software, permitiendo modificaciones de este en caliente.

En resumen, la interfaz VICI surge como intento de mejorar la situación para los integradores de sistemas, al proporcionar una interfaz estable, que permite a las herramientas externas consultar, configurar y controlar el demonio IKE de una manera simple y efectiva. Dentro de los servicios que usan esta interfaz, destaca *swanctl* [31], una aplicación que se usa en línea de comandos y nos permite configurar, consultar y en definitiva, controlar el demonio de *charon*. Para hacer uso de esta interfaz en nuestro trabajo, la usaremos dentro del código del módulo que hemos diseñado, por lo que tienen que estar en la misma máquina tanto la implementación del módulo como la de VICI.

2.6.2. Arquitectura y funcionamiento

VICI implementa un protocolo IPC para configurar, monitorear y controlar el demonio de IKE, *charon*, utilizando mensajes de solicitud y respuesta, junto a eventos para comunicarse a través de un transporte basado en un flujo seguro, implementando la parte del servidor. Para ello, se abre un *socket* al cual se conecta el cliente, accediendo así a la funcionalidad. El protocolo de transporte está segmentado, y cada secuencia tiene una cabecera de 32 bits. La secuencia de bytes de recepción debe ser la misma que la de envío, es decir, no debe alterarse el orden en que llegan los paquetes al receptor.

Dentro de la capa de transporte, cliente y servidor pueden intercambiar paquetes, donde el tipo de paquete define su contenido y estructura, y este es un identificador de 8 bits situado al principio del paquete. Actualmente tenemos definidos los siguientes mensajes: *CMD_REQUEST*, *CMD_RESPONSE* y *CMD_UNKNOWN*

(0, 1 y 2 respectivamente, correspondientes a mensajes de solicitud, de respuesta y de respuesta si no se conoce el comando solicitado), *EVENT_REGISTER* y *EVENT_UNREGISTER* (3 y 4 respectivamente, asociados al registro y la cancelación de registro de un evento concreto), *EVENT_CONFIRM*, *EVENT_UNKNOWN* y *EVENT* (5, 6 y 7 respectivamente, atribuidos a confirmar un registro o cancelación, a informar de un fallo en registro o cancelación y finalmente a un mensaje de evento concreto).

Con respecto a los comandos, son siempre emitidos por los clientes y solo un comando puede estar activo en un momento determinado por conexión (debido a la carencia de un número que asocie las respuestas a las solicitudes). Los eventos se pueden generar en cualquier momento mientras la conexión esté activa y el cliente esté registrado, incluso durante un comando activo.

Cada mensaje que se intercambia usa un árbol jerárquico, donde cada sección contiene un conjunto arbitrario de pares *clave/valor*, listas y otras secciones. Cada elemento de estas secciones contiene un tipo de elemento y un nombre o valor del elemento. Cada tipo está codificado con un valor de 8 bits y una cadena que sigue al tipo indicando el valor de un atributo en caso de que sea necesario. Los tipos de elementos de un mensajes son: *SECTION_START* y *SECTION_END* (valores 1 y 2, indicando el comienzo y el fin de cada sección), *KEY_VALUE* (indicando un valor concreto de una clave, con valor 3), y por último *LIST_START*, *LIST_ITEM* y *LIST_END* (con valores 4, 5 y 6 respectivamente, indicando el comienzo de una lista, un valor dentro de una lista y el final de una lista).

Por parte de las secciones, estas se pueden abrir en cualquier punto del mensaje y se pueden anidar y los mensajes de *SECTION_START* y *SECTION_END* deben aparecer el mismo número de veces. En el caso de los pares *clave/valor*, pueden aparecer en cualquier lugar del mensaje y deben ser únicos en la sección que se encuentren. Finalmente, las listas no pueden estar anidadas y los elementos solo pueden aparecer dentro de las listas, permitiéndose listas vacías, y teniendo que aparecer el mismo número de veces *LIST_START* y *LIST_END*.

Es importante destacar en la representación e interpretación de los mensajes, que cada valor puede tomar valores arbitrarios y diferentes según la conversión que haga cada aplicación de esta información, puesto que no está definido en ningún sitio una interpretación uniforme. Una vez explicados todos los componentes de una estructura de un mensaje, vamos a ver un pequeño ejemplo a continuación, y luego veremos en más detalle como está codificado a bajo nivel la representación de la correspondiente información [32]:

```
key1 = value1
section1 = {
```

```

        sub-section = {
            key2 = value2
        }
list1 = [ item1, item2 ]
}

/* key1 = value1 */
3, 4, 'k', 'e', 'y', '1', 0, 6, 'v', 'a', 'l', 'u', 'e', '1',
/* section1 */
1, 8, 's', 'e', 'c', 't', 'i', 'o', 'n', '1',
/* sub-section */
1, 11, 's', 'u', 'b', '-', 's', 'e', 'c', 't', 'i', 'o', 'n',
/* key2 = value2 */
3, 4, 'k', 'e', 'y', '2', 0, 6, 'v', 'a', 'l', 'u', 'e', '2',
/* sub-section end */
2,
/* list1 */
4, 5, 'l', 'i', 's', 't', '1',
/* item1 */
5, 0, 5, 'i', 't', 'e', 'm', '1',
/* item2 */
5, 0, 5, 'i', 't', 'e', 'm', '2',
/* list1 end */
6,
/* section1 end */
2,

```

Podemos observar, por ejemplo en la primera línea codificada, como se indica un 3 (que representaba a la entrada de un valor) seguido de un 4 (que representa la longitud del nombre) y los 4 bytes correspondientes. Después observamos un 0 (al no estar asociado a ningún apartado especial de mensaje, se supone simplemente valor de un atributo) junto a un 6 (indicando que el valor ocupa 6 bytes) y los bytes correspondientes. Otro ejemplo puede ser la línea en la que se crea una lista, donde se indica el valor 4 (que representaba el comienzo de una lista, *LIST_START*) y luego se indican los 5 bytes que ocupara el nombre, junto a los bytes correspondientes. La explicación de estos dos mensajes es equivalente para el resto de mensajes que se muestran en la parte superior y para todos los mensajes que emplea VICI.

2.6.3. Comandos

Sobre la capa de mensajes, VICI utiliza *CMD_REQUEST* y *CMD_RESPONSE* para implementar los comandos que solicita el cliente y responde el servidor. Un cliente debe registrarse en el evento apropiado para recibir la transmisión de la información solicitada y cancelar el registro después de que se haya recibido la respuesta completa. Tenemos una gran cantidad de mensajes definidos y disponibles para usar [32], y vamos a destacar aquí los que consideramos más relevantes:

- *stats()*: Nos devuelve una lista de las estadísticas y la información que posee el demonio de IKE en ese momento.

- *initiate()* & *terminate()*: Estos dos comandos nos ofrecen la posibilidad de iniciar y terminar una asociación de seguridad, transmitiendo eventos de registro de control.
- *rekey()*: Usado para recargar una asociación caducada.
- *redirect()*: Redirige a un cliente a otro enlace, disponible solo para IKEv2 y si lo soporta el correspondiente *peer*.
- *install()* & *uninstall()*: Nos permiten instalar y desinstalar diferentes políticas a la configuración de una asociación de seguridad, como por ejemplo *bypass*.
- *load-conn()* & *unload-conn()*: Esta pareja de comandos nos da la opción de cargar una conexión definida dentro del demonio (si ya existe una conexión con ese nombre la reemplaza por la nueva), o la eliminación de esta.
- *load-shared()* & *unload-shared()*: Nos dan la posibilidad de cargar un secreto compartido previamente (en nuestro caso usaremos el secreto *psk*, *pre-shared key*) en el demonio, o la posibilidad de eliminarlo de este.

En el desarrollo del trabajo usaremos una implementación en *Python* del protocolo VICI, orientado a la implementación de aplicaciones cliente. El módulo nos proporciona un constructor *Session()* para una interfaz de alto nivel, proporcionando métodos para los comandos VICI admitidos. Para facilitar la representación de los mensajes en el árbol de datos que usa VICI, esta implementación convierte los árboles en estructuras de datos en *Python*, y la clase *Session* nos devuelve y envía objetos de este tipo.

2.7. Contextualización del trabajo

El desarrollo de este trabajo de fin de grado está englobado en el contexto de anteriores trabajos relacionados con la gestión de configuraciones de seguridad en SDN, destacando el trabajo de fin de grado *Configuración centralizada de IKE en múltiples dispositivos con el uso de SDN* [33].

El desarrollo del trabajo de fin de grado previo se centraba principalmente en el objetivo de diseñar e implementar un módulo que permita a los dispositivos ubicados en el plano de datos la instanciación de la configuración recibida por el controlador a través de NETCONF. Por lo tanto, el módulo debe ser capaz de configurar las *database* del dispositivo, las SPD y SAD, y finalmente el demonio de IKE. La realización del módulo también proporciona la ventaja de poder aplicar las configuraciones en caliente, sin necesidad de usar otros ficheros ni tener que reiniciar servicios. El desarrollo del trabajo deja a un lado las comunicaciones realizadas por las interfaces *Northbound*, *Eastbound* y *Westbound*, centrándose especialmente en la interfaz *Southbound* con el uso de NETCONF, dejando fuera del ámbito de estudio el nivel de aplicación o la comunicación con otros controladores. Para realizar la prueba de funcionamiento, se simularon virtualmente los dispositivos necesarios y las redes implicadas en el escenario, descritos con detalle en la sección 2.7.2, donde se observan los diferentes escenarios que teníamos disponibles en el punto de partida del trabajo (observando tanto los dispositivos como las redes necesarias), y se demostró el funcionamiento introduciendo nuevas políticas desde el controlador a los diferentes nodos.

Como se puede observar en el trabajo, el objetivo se centra en la seguridad, intentando dar al escenario una alta capacidad de gestión y de configuración de asociaciones de seguridad entre cualquier par de dispositivos de la red del plano de datos, siendo un objetivo conjunto al desarrollo de nuestro trabajo.

Queda en evidencia la relación entre los dos trabajos de fin de grado, donde el principal objetivo del desarrollo de nuestro trabajo se basa en la mejora del trabajo anterior sobre el que partimos. Para ello, el desarrollo del módulo se realiza con *Python*. También hacemos uso dentro del módulo de una implementación para *Python* de la interfaz VICI, permitiéndonos la instanciación de políticas directamente al *kernel* en caliente. En relación a los *callbacks*, hacemos uso de la segunda versión de NETCONF usando la librería *libyang*. El principal avance respecto al trabajo anterior se centra en la elaboración de un nuevo módulo y el uso de *Python* para la implementación del módulo, ya que la implementación previa de estos posee una mayor complejidad y una mayor carga de trabajo a la hora de procesar y cargar las diferentes configuraciones, sumado a que la realización del módulo se implementó en lenguaje C.

2.7.1. Vías de desarrollo

La realización y el desarrollo del trabajo se centran en la gestión de asociaciones de seguridad, donde surgen dos vías de desarrollo principales, explicadas con más detalle en el documento [34], en el cual se especifican dos casos diferentes para la protección de los flujos de datos IPsec.

En el primer caso, se realiza haciendo uso de un protocolo de negociación inicial de claves, haciéndose uso de IKEv2. El controlador se encargará de gestionar y aplicar entradas de las bases de datos SPD y PAD del dispositivo gestionado. También necesita manejar información que usará IKE para la negociación de claves, como una clave compartida previamente. De esta manera, IKE se encarga de establecer las asociaciones de seguridad IPsec entre los dispositivos correspondientes. Los servicios de protección de flujo IPsec basados en SDN proporcionan una gestión dinámica y flexible de las asociaciones, facilitando la labor del administrador. Para poder obtener esta gestión, se requiere al menos un modelo de datos YANG tanto para datos de estado como de configuración. Adicionalmente, en caso de haber varios controladores, es recomendable tener algún protocolo o mecanismo que ayude a descubrir que controlador maneja un dispositivo concreto. En la elaboración del trabajo hemos partido de este primer caso.

En el segundo caso, la gestión se realiza sin disponer de ningún protocolo que realice la negociación inicial de las claves ni las asociaciones de seguridad. En este caso, será el controlador el encargado de gestionar las bases de datos SPD y SAD (se gestiona la SAD puesto que el controlador configura directamente las asociaciones de seguridad y no delega en ningún protocolo), eliminando la gestión de PAD puesto que no se hace uso de IKE. Al igual que en el primer caso, para implementar este segundo caso se necesita al menos un modelo de datos YANG tanto para datos de estado como de configuración, unido a la recomendación de tener algún protocolo o mecanismo que ayude a descubrir que controlador maneja un dispositivo concreto, para la situación de tener más de un controlador. Este segundo caso será objeto de estudio en otro trabajo de grado, desarrollado de forma paralela a este.

El escenario que se va a usar para el desarrollo es el mismo que se exponía en la sección 2.1.3, aunque en este caso el flujo de datos y de información será diferente. En este caso, mostramos el escenario *host-to-host* (escenarios que veremos en más detalle en la siguiente sección), donde el escenario consta de un controlador y dos *hosts* finales, y el controlador se encarga de instanciar la configuración para proteger el tráfico entre ellos. El administrador será el encargado de introducir en el controlador la información relacionada con la asociación de seguridad correspondiente, y este comunicará la configuración mediante la interfaz *Southbound* a los dos nodos. Cuando se hayan configurado correctamente, crearán entre los dos nodos un canal seguro IPsec basado en la información que han recibido, y también comunicarán al controlador que se ha realizado correctamente la aplicación de las políticas de

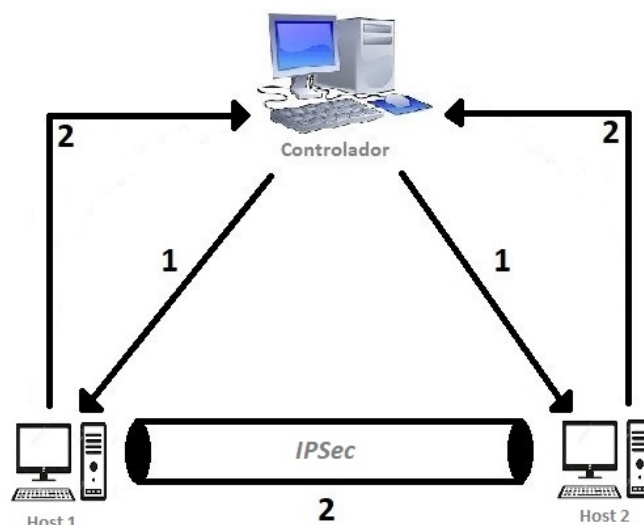


Figura 13: Aplicación de asociaciones de seguridad.

seguridad. Este funcionamiento se aplica a ambos casos presentados antes.

Podemos observar todo lo comentado antes en la Figura 13, donde el primer paso se corresponde al momento en que el administrador indica al controlador que los *hosts* 1 y 2 deben proteger el tráfico entre ellos, y envía la información necesaria para aplicar la asociación de seguridad. El segundo paso se corresponde con la notificación al controlador de la correcta aplicación de la asociación, y finalmente la creación de un canal seguro para el tráfico que se intercambie entre los *hosts*.

2.7.2. Escenario desarrollado

En este apartado, vamos a exponer las tres distribuciones de escenarios que se nos han proporcionado al inicio del desarrollo del proyecto, comentando los aspectos principales de cada uno (se puede ver una información más desarrollada y los pasos para lanzar los distintos escenarios mostrados a continuación en [35]). Vamos a ver los escenarios a un bajo nivel de detalle, centrándonos en los dispositivos y su distribución más que en el direccionamiento y las redes, y luego en el apartado de desarrollo de la solución veremos el escenario seleccionado en más detalle.

En primer lugar, tenemos el escenario *host-to-host*, como se observa en la Figura 14. Esta topología dispone de tres elementos principales, donde tenemos un equipo actuando de controlador y dos equipos que actuarán como *hosts* o nodos de la red. En el escenario disponemos de dos redes principales, donde por un lado tenemos la red sobre la que el controlador configura y se comunica con los nodos

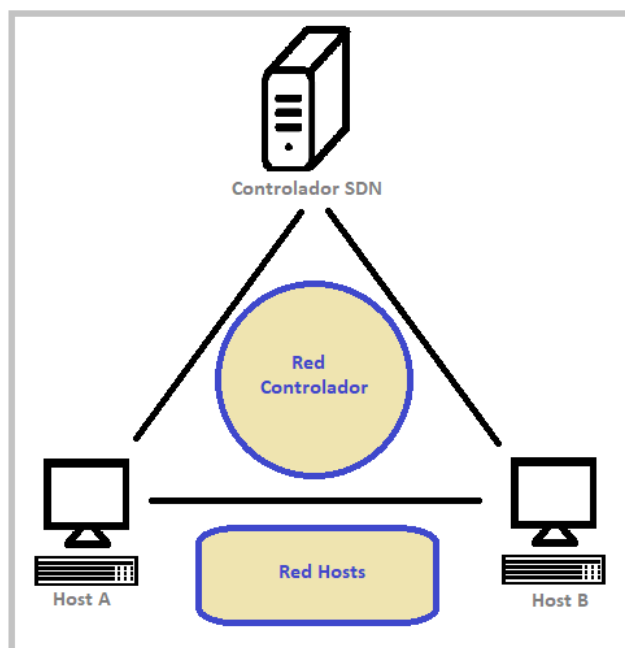


Figura 14: Topología del escenario *host-to-host*.

(red controlador), y por otro lado tenemos la red que comunica los diferentes nodos finales de la red (red hosts). Las comunicaciones realizadas en la red controlador se realizarán mediante NETCONF e irán protegidas por TLS o SSH, mientras que la comunicación en la red de los nodos (una vez protegido el tráfico con la correspondiente asociación de seguridad) irá protegida con IPsec.

Este es el escenario que hemos seleccionado para la realización de las pruebas del caso de uso final. Para la protección del tráfico IPsec se ha usado mayoritariamente el modo transporte, puesto que es el más indicado entre dos nodos finales, aunque también se han realizado pruebas con el modo túnel.

Como segunda opción de distribución de escenario, tenemos la topología *gateway-to-gateway*, dispuesta tal y como se ve en la Figura 15. En ella, observamos la aparición de al menos cinco elementos destacables. En primer lugar, en la parte superior podemos observar al figura del controlador. En la parte inferior, disponemos de dos *hosts* finales en sus respectivas redes, que a su vez están conectados cada uno a un nodo encargado de dirigir el tráfico.

En esta topología, disponemos de cuatro redes. En primer lugar tenemos la red sobre la que se comunica el controlador con los diferentes nodos (red controlador) sobre la que usaremos NETCONF. En la parte inferior, tenemos las redes propias de cada *host* (red A y red B) y una red que une los diferentes nodos enrutadores de

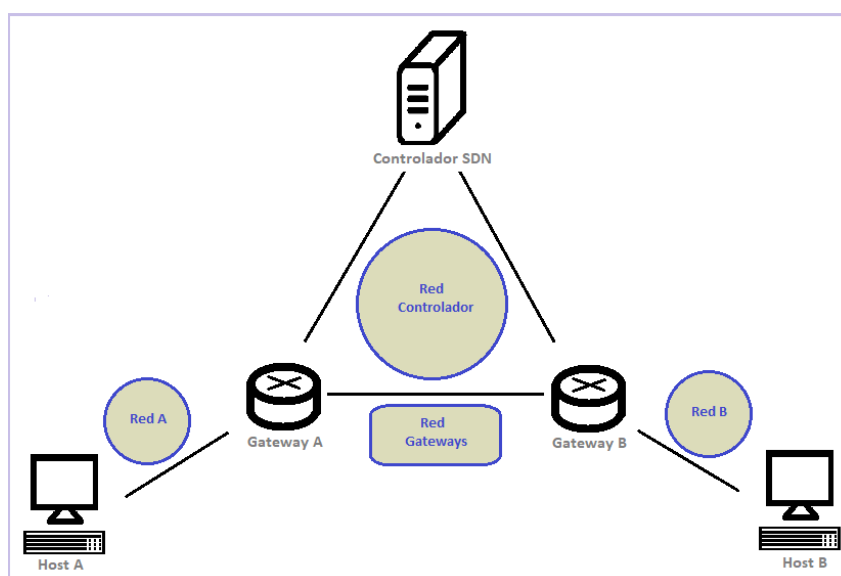


Figura 15: Topología del escenario *gateway-to-gateway*.

la red (red *gateways*).

Esta topología se corresponde con la más común en escenarios reales, aunque hemos seleccionado el escenario *host-to-host* por simplicidad a la hora de montar el escenario y configurarlo. Con respecto a las comunicaciones de las diferentes redes, la red controlador funcionará sobre TLS o SSH. La red en la que se comunican los distintos *routers* funcionará principalmente en modo túnel, y las comunicaciones dentro de las redes donde se ubican los dispositivos finales se hará usando modo transporte (se pueden variar los modos, pero indicamos el escenario más común).

En último lugar, disponemos de la topología *n-hosts* observada en la figura Figura 16. En ella, tenemos en primer lugar un controlador y en segundo lugar una serie de nodos o *hosts* (aunque en la imagen se muestren cuatro, realmente podría ser una cantidad diferente de estos). Se observan dos redes diferentes, donde tenemos en primer lugar una red entre el controlador y los diferentes nodos de la topología, en la que el controlador gestionará los nodos usando NETCONF, llamada red controlador. Por otra parte, tenemos la red sobre la que se comunicarán los diferentes nodos finales de la red, llamada red *hosts*.

Este escenario se puede considerar el más complejo, por lo que indicamos de nuevo que se ha seleccionado el escenario *host-to-host* por simplicidad. Con respecto a las comunicaciones, al igual que en el resto de casos, la comunicación entre el controlador y los *hosts* en la red controlador se realiza usando TLS o SSH. Por parte de la red *hosts*, se usará IPsec una vez se hayan establecido las correspondientes aso-

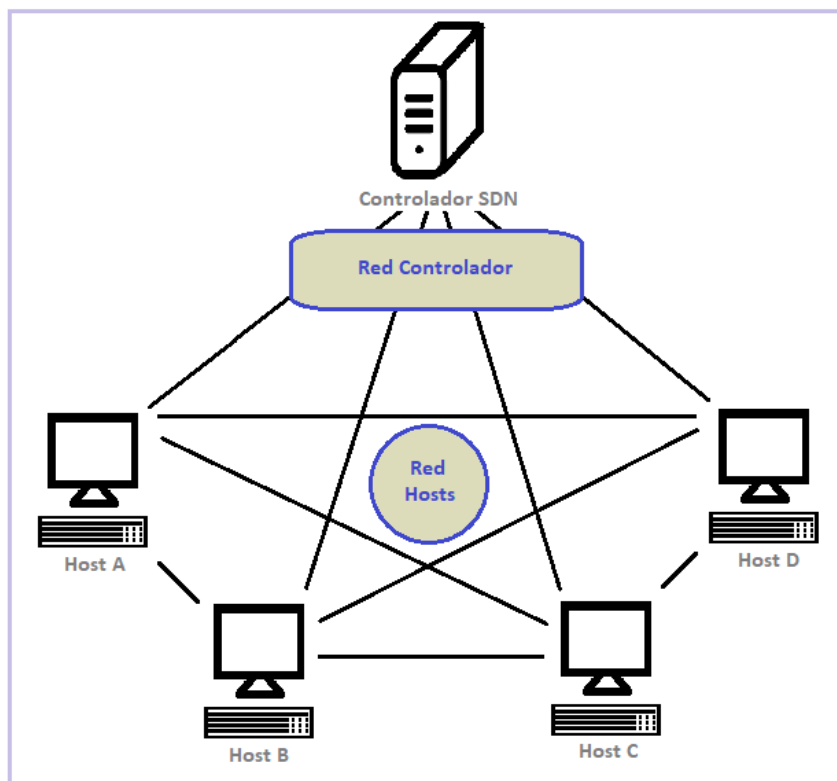


Figura 16: Topología del escenario *n-hosts*.

ciaciones de seguridad, funcionando tanto en modo túnel como en modo transporte en función de las necesidades de la red.

3. Objetivos y metodología

A lo largo de este apartado se definen los objetivos y la metodología empleada en el desarrollo de este documento. Con respecto a los objetivos del proyecto, la idea general consiste en el desarrollo de un módulo que se ejecute en los dispositivos de red y automatice la aplicación de configuraciones de seguridad. En primer lugar, vamos a comentar con detalle los principales objetivos a cubrir con el proyecto, y posteriormente comentaremos paso a paso la metodología empleada para ello.

Como primer objetivo tenemos la indagación en los diferentes escenarios y topologías donde puede ser aplicado el paradigma SDN, valorando en cada caso si es realmente ventajoso su uso. También se hará un estudio y análisis de las principales tecnologías de las que hace uso este paradigma, en especial las que usaremos a lo largo del proyecto (que son las tecnologías comentadas en el apartado anterior), investigando a través de diferentes fuentes, como por ejemplo el IETF (*Internet Engineering Task Force*).

El escenario requiere también un modelo de datos que pueda albergar toda la información de los protocolo IPsec e IKE, modelo que ha sido dado en el punto de partida y no entra en la competencia del desarrollo de nuestro trabajo. Este modelo de datos debe ser válido para todas las posibles configuraciones que se quieran realizar sobre los distintos nodos, pudiendo indicar en este diferentes parámetros o valores, sin que requiera un cambio en el modelo, y que pueda ser comprendido por todos los elementos de la topología que requieran su uso. En el desarrollo de este proyecto la información intercambiada en los mensajes irá codificada en formato XML.

Otro objetivo fundamental pasa, en primer lugar por la comprensión del trabajo realizado por cada nodo para instanciar la asociación de seguridad (información que recibe en un mensaje codificado con el modelo de datos previamente mencionado), y en segundo lugar por conseguir que los nodos realmente puedan decodificar, comprender y aplicar la configuración que reciben del controlador. En conjunto, debemos crear una implementación de un módulo ejecutable en cada nodo (programada en *Python*), que maneje correctamente las configuraciones que negocia IKE, a través del uso de NETCONF y validando frente a los modelos YANG necesarios. Se crearán las políticas de asociación para un número determinado de nodos, y mandará la configuración a cada nodo (ajustada en función del nodo correspondiente), y estos deben configurarse para establecer una comunicación segura en función de los parámetros que ha indicado el controlador.

Teniendo la información en cada nodo y habiendo sido estos capaz de comprenderla, usarán la interfaz VICI para instanciar los cambios directamente en el *kernel* del sistema operativo, sin necesidad de tener que reiniciar o paralizar el compor-

tamiento previo. Es decir, que la interfaz nos permite aplicar las políticas en el momento que las recibamos, consiguiendo una alta eficiencia a la hora de aplicar los cambios. A partir de este punto, se establece una comunicación segura entre los nodos que ha configurado el controlador.

Por último, se desarrollará una prueba de concepto a modo de demostración, en la que se mostrará el correcto funcionamiento de todos los objetivos presentados. En esta prueba podremos ver la instanciación de políticas de seguridad relacionadas con el protocolo ESP, donde podremos ver configuraciones tanto en modo túnel como en modo transporte, cambiando también algunos valores como el tiempo necesario para volver a generar las claves de una comunicación. Todas las pruebas se harán en un escenario virtual previamente montado, el cual simulará los dispositivos físicos que componen la topología.

A continuación, vamos a presentar en un orden cronológico la metodología usada para la consecución de los objetivos presentados:

- Como primer paso, es fundamental comprender el paradigma de SDN y la motivación que ha llevado a su uso, ya que en él se basa todo el desarrollo del proyecto. La comprensión de este paradigma lleva a su vez a la necesidad de entender las diferentes tecnologías e interfaces que usa para ofrecer su correcto funcionamiento.
- En este punto indagaremos sobre que tecnologías son realmente necesarias o cuales nos ofrecen mejor funcionamiento o características, de cara a la realización del proyecto concreto. Se harán pruebas con las diferentes tecnologías seleccionadas, hasta quedarnos con las que nos sean realmente útiles.
- Comprendida la topología, es necesario crear el escenario sobre el que se va a trabajar. Este escenario se basa en la topología *host-to-host*, donde tendremos en el plano de control un único controlador y en el lado del plano de datos dos nodos. Recordemos que el despliegue final de este escenario será virtual y simulará el comportamiento de un escenario real.
- Una vez tenemos funcionando el escenario sobre el que vamos a trabajar, es necesario crear los ficheros que aplicaremos a ambos nodos, que contendrán las asociaciones de seguridad. En estos fichero irá toda la información necesaria para que ambos nodos puedan establecer una comunicación segura basada en IKEv2, creándose manualmente en el controlador.
- Cuando tengamos los ficheros disponibles, debemos desde el controlador instanciar el fichero correspondiente en cada nodo, usando *Netopeer*, y que estos nodos apliquen la configuración que el fichero dicta. En este punto, la comunicación entre los distintos nodos instanciados estará protegida en base a la política indicada.

- En base a todo lo comentado, se ejecutará una prueba de concepto con el fin de demostrar el correcto funcionamiento y la consecución de los objetivos descritos.
- Ampliaremos el escenario inicial de ESP (funcionando en modo transporte) para permitir el funcionamiento en modo túnel, y cambiar también opcionalmente el tiempo de expiración de las claves. Aunque pondremos estos ejemplos de cambios en las políticas a aplicar, no implica que no se puedan realizar otras configuraciones diferentes a las indicadas por nosotros.
- Será necesaria una validación de todas las pruebas de concepto realizadas, valorando el cumplimiento o no de los objetivos, junto a posibles mejoras o calidad de las pruebas.
- Por último, se hará un análisis sobre la situación actual de la investigación en este ámbito, indicando las posibles ramas o vías de investigación que quedan abiertas para el futuro.

4. Diseño de la solución

De los objetivos que planteamos en el desarrollo del proyecto, hemos comprendido la necesidad del paradigma SDN y estudiado las distintas tecnologías que usaremos junto a este para el desarrollo del proyecto, siendo en este punto cuando toca indagar en el desarrollo de una solución.

Durante el desarrollo del diseño del proyecto, surgen dos opciones principales relacionadas con la instanciación de asociaciones de seguridad, comentadas en la sección 2.7.1. De ellas seleccionamos la primera opción usando un protocolo de intercambio de claves, en este caso IKE, para la instanciación de claves para la creación de un canal seguro entre nodos, intercambiando directamente el material criptográfico necesario para el establecimiento del canal IPsec. Se va a comentar el detalle el funcionamiento de este caso en el siguiente apartado.

4.1. Administración con IKE en el dispositivo

Como introducíamos, vamos a hacer uso del escenario que implementa un protocolo de intercambio y distribución de claves, donde cada elemento de la topología implementa IPsec junto a IKE (en concreto IKEv2), intercambiando las distintas configuraciones de las asociaciones de seguridad a través del protocolo NETCONF. Estos archivos contienen información relacionada con SPD y PAD, que son las bases de datos que se deben modificar para almacenar la información, junto a la información necesaria para crear una negociación de claves con IKE. Una vez realizada la negociación IKE, se almacenará la información correspondiente en la SAD (hablamos de la finalidad de cada base de datos que estamos comentando en la sección 2.3.2).

Para el correcto desarrollo de la solución necesitamos cumplir unos requisitos derivados del escenario comentado. Se requiere en primer una entidad que haga de controlador capaz de generar asociaciones de seguridad y aplicarlas en los correspondientes nodos destino, junto a otro par de entidades que actúen de *hosts* finales y sean capaces de interpretar la información relacionada con las asociaciones de seguridad, y aplicarlas. También se requiere que las entidades sean capaces de usar los protocolos de seguridad IPsec e IKE, junto al protocolo NETCONF encargado de comunicarse entre controlador y *hosts*, pudiendo manipular las diferentes políticas de seguridad.

4.1.1. Establecimiento de canal seguro

Vamos a ir analizando el flujo de mensajes que se produce dentro del escenario siguiendo la Figura 17, donde todo el flujo de mensajes, tal y como hemos comentado anteriormente, se desarrolla en la interfaz *Southbound*.

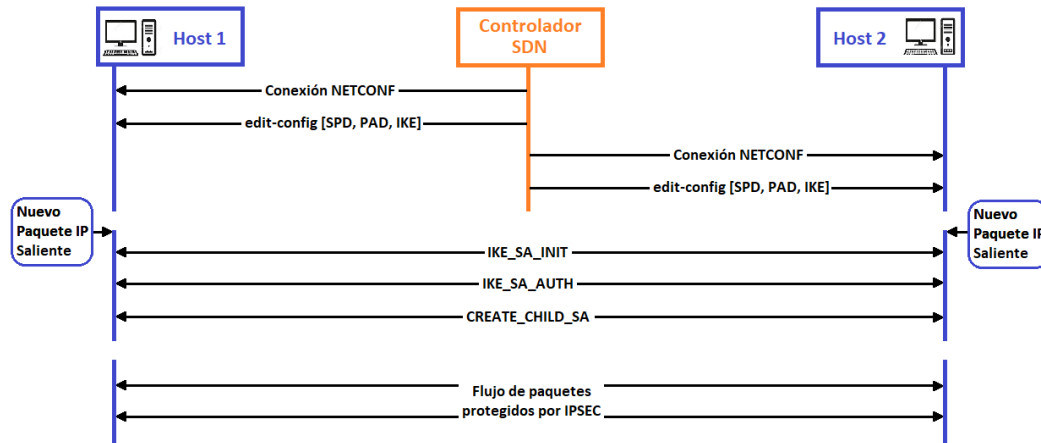


Figura 17: Flujo de mensajes en escenario *host-to-host*.

En primer lugar, el controlador (que posee la información necesaria para cada *host*) crea una conexión con el *host 1*. Una vez establecida la conexión, envía la información en un archivo XML para que almacene la información y pueda crear un canal seguro con el *host 2*, usando para enviar la configuración el comando *edit-config* y enviando con este comando el archivo XML. Una vez concluida la transferencia del archivo, se cierra la conexión y se crea otra con el *host 2*, procediendo de igual manera que antes, enviándole un archivo XML con la configuración necesaria para poder crear a continuación una conexión segura con el primer *host*, y finalmente cerrando la conexión. El fichero XML que se envía a cada *host* contiene toda la información necesaria para la creación del canal seguro, es decir, la información necesaria de SPD, PAD e IKE.

Otra forma diferente y más óptima sería crear una conexión con ambos *hosts* a la vez, enviado simultáneamente la información necesaria y cerrando la conexión al concluir la transferencia (en puesto de proceder para cada uno por separado), donde todo esto estaría optimizado ya en el controlador.

Una vez poseen ambos *hosts* la información necesaria para crear el canal seguro, uno de estos quiere enviar un nuevo paquete al otro. En este momento, el *host* que envía el paquete se da cuenta que en su SAD no posee ninguna regla IPsec para su procesamiento, motivo por el cual delega en IKE la negociación de claves para establecer el canal. En este momento, suceden los mensajes iniciales para la negociación de IKE (mensajes que comentamos con más detalle en la sección 2.4.2), iniciados por el *host* que quiere enviar un paquete al otro. En primer lugar se negocian los algoritmos criptográficos con *IKE_SA_INIT*, después autentica los mensajes del paso anterior e intercambian identidades con *IKE_SA_AUTH*, y finalmente crea una

nueva asociación de seguridad con *CREATE_CHILD_SA*. Al terminar el proceso de IKE, ambos *hosts* tienen en su SAD la información necesaria para procesar los paquetes entre ellos.

Una vez ha acabado la negociación de IKE, ahora si dispone el *host* que quiere enviar el paquete de una entrada en la SAD, y usando la información que posee establece un canal seguro entre ambos. A partir de este punto, todo el tráfico de paquetes que se envíen entre ellos estarán protegidos.

4.1.2. Funcionamiento interno del establecimiento

Comprendido el flujo de mensajes que se produce durante el establecimiento del canal seguro, vamos a indagar a más bajo nivel los diferentes intercambios de información que se producen, no solo entre controlador y *host*, sino también entre los propios elementos internos de cada uno de ellos. Para ello, mostramos la Figura 18, donde detallamos los pasos que se van siguiendo, y vamos a ir comentándolos paso a paso. En ella, hay que aclarar que se muestra un controlador y un dispositivo de red (en nuestro caso real será un *host*), puesto que procederá de manera similar para cualquier dispositivo que se encuentre en la red. Tanto el controlador como los dispositivos necesitan tener funcional el protocolo NETCONF, donde tal y como hemos comentado previamente, el controlador hará de cliente y los dispositivos de red controlados jugarán el papel de servidor.

En primer lugar, el administrador de la red debe aplicar unas políticas específicas desde el plano de aplicación, en un lenguaje de alto nivel, a través de la interfaz *Northbound* (paso 1), para ser aplicadas en el momento que se requiera una comunicación segura entre algunos de los dispositivos que controla gestiona. Cuando el controlador posee la información, este debe comunicarse con todos los dispositivos que estén involucrados en esas políticas.

Para comunicarse, inicia una sesión con el dispositivo correspondiente y carga la configuración relacionada con SPD, PAD e IKE (paso 2), toda la información codificada en un archivo XML. No contiene información relacionada con SAD puesto que se delegará en IKE para la generación de las claves. La creación de la sesión y el envío de la configuración se delega en el protocolo NETCONF, comenzando en el cliente y comunicándose con el servidor. Una vez el dispositivo ha recibido la información, se verifica el mensaje frente al módulo YANG correspondiente, y si la verificación es correcta, se cargarán los datos de la conexión y del secreto compartido en IKE (paso 3). En el momento que IKE posee la información correspondiente a las bases de datos, las aplica directamente en cada una de ellas, tanto en PAD (que la simula la propia implementación de *StrongSwan*) como en SPD (paso 3b). La nueva información se aplica sin necesidad de reiniciar ningún servicio, y se notifica al controlador cuando la configuración se haya aplicado correctamente.

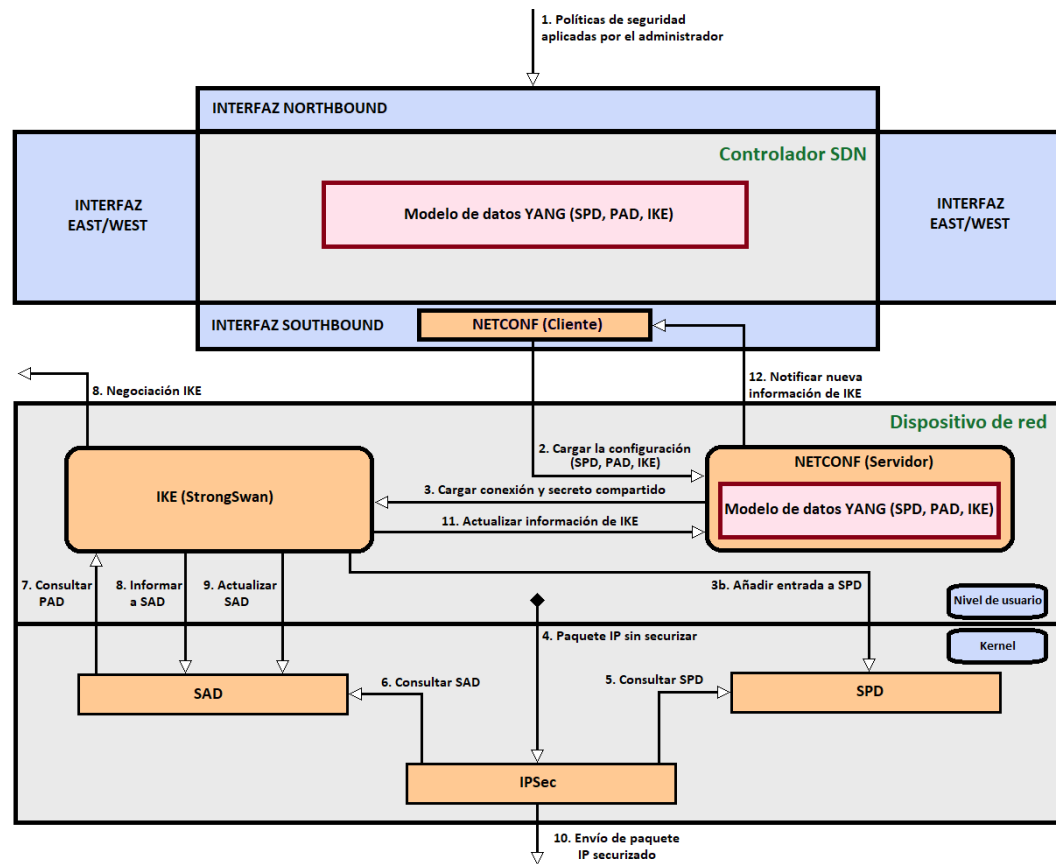


Figura 18: Funcionamiento interno del establecimiento de asociaciones.

Cuando el dispositivo quiere enviar un paquete a otro, se pasa al *kernel* el paquete para que sea tratado por IPsec (paso 4). En este punto IPsec debe comprobar frente a SPD la política a aplicar al paquete, y cuando encuentra una coincidencia, aplica la política que haya en la entrada (paso 5). En este ejemplo, la política a aplicar es *protect*, donde se debe procesar el paquete y protegerlo, pudiendo ser también descartado o dejado avanzar, *discard* y *bypass* respectivamente (visto en más detalle en la sección 2.3.2). Como en este caso la política de SPD indica que hay que securizar el paquete, se consulta la SAD para saber que tipo de seguridad se le va a aplicar al paquete (paso 6).

En este punto, la SAD comprobará si posee entradas que coincidan con el paquete, no encontrando ninguna en este caso concreto. Al no localizar coincidencia, se debe consultar a IKE para que genere unas nuevas claves con el dispositivo destino. Este comienza una negociación con el dispositivo destino del paquete, e informa a SAD y crea un estado nuevo en ella (pasos 7 y 8). Cuando la negociación ha fina-

lizado, se actualiza la SAD con los nuevos valores obtenidos con IKE (paso 9), y ahora cuando se comprueba si hay alguna coincidencia con el paquete, esta si contiene una entrada. Se aplica la protección que indique la entrada y, finalmente, se envía el paquete a su destino (paso 10). Acto seguido, si la negociación de IKE se ha desarrollado correctamente, se actualiza la información de IKE frente a NETCONF (paso 11), y a su vez este notifica a la instancia de NETCONF del controlador los cambios que ha realizado (paso 12).

4.2. Funcionamiento interno del dispositivo de red

Una vez comprendido el funcionamiento interno de los dispositivos finales cuando reciben una nueva configuración, es fundamental indagar en más profundidad sobre el funcionamiento interno que posee el propio servidor NETCONF, ya que hacemos uso de tecnologías e implementaciones que merecen ser nombradas, y dejando también claro la posición que ocupa el módulo que hemos creado en el procesamiento de las configuraciones recibidas, quedando el funcionamiento explícito del módulo lo detallado en la sección siguiente. Partiendo de la figura que mostrábamos en el apartado anterior, vamos a focalizar el funcionamiento del servidor NETCONF, usando para la explicación la Figura 19.

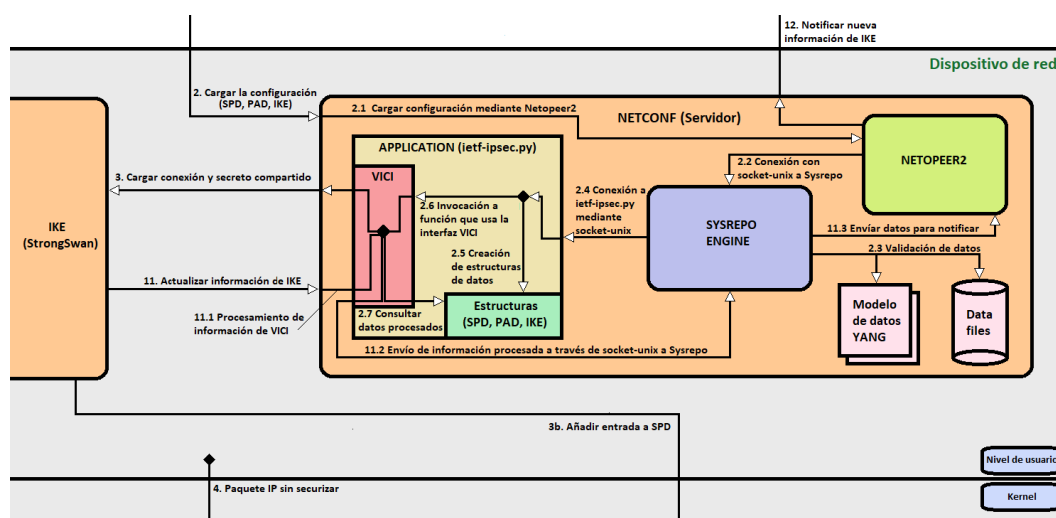


Figura 19: Funcionamiento interno del servidor NETCONF.

Antes de proceder a explicar los pasos realizados cuando se recibe una nueva configuración, vamos a detallar los diferentes elementos que están operativos dentro del servidor de NETCONF. En primer lugar, aparece nuestro módulo, en el cual destaca la aparición de VICI y de unas estructuras que hemos diseñado, comentadas también en la siguiente sección. Por otra parte, aparece *Sysrepo* y sus correspondientes archivos frente a los que comprueba las configuraciones recibidas (en este caso el modelo de datos YANG y los archivos de datos).

4.2.1. Sysrepo

Sysrepo [36] aparece con el objetivo de solventar una importante problemática, centrada en la necesidad de unificar en una sola implementación los ficheros que usan diferentes aplicaciones y demonios a la hora de la configuración. De esta manera, permite a los desarrolladores centrarse en otro tipo de funcionalidad y delega

en *Sysrepo* la configuración y los datos operativos. *Sysrepo* es un almacén de datos operativo basado en YANG para aplicaciones Unix, donde las aplicaciones pueden usarlo para almacenar configuración modelada por YANG en lugar de usar ficheros de configuración plana. Por ultimo, garantizará la consistencia de los datos y aplicará las restricciones definidas por el modelo YANG. Funcionando con la ayuda de *Netopeer*, las aplicaciones se podrán administrar también de forma remota usando NETCONF.

4.2.2. *Netopeer2*

Netopeer [37] se define como una selección de herramientas del protocolo NETCONF, construido en la librería *libnetconf*. La idea de *Netopeer* es simplificar la conexión entre diferentes dispositivos que tengan implementado el protocolo NETCONF, entre clientes y servidores. Recordamos que en este protocolo, implementan la parte de servidor los dispositivos a los que se conecta el controlador para aplicar diferentes configuraciones y la parte de cliente el propio controlador. Con el paso del tiempo, se movieron las actividades para trabajar en una segunda generación de este conjunto de herramientas, en este caso basadas en la biblioteca *libyang* [38] y usando *Sysrepo* con implementación de almacén de datos. Esta segunda versión ha sido llamada *Netopeer2* [39], y está lo suficientemente desarrollada como para ser utilizado como reemplazo de la primera versión.

Por lo tanto, *Netopeer2* es la versión que usamos, y nos da la posibilidad de simplificar la conexión entre diferentes dispositivos que tengan implementado el protocolo NETCONF, permitiéndonos la configuración de forma remota.

4.2.3. Procesamiento interno de configuración

Una vez teniendo comprendidas las principales entidades, vamos a comentar los pasos que se suceden una vez llega una nueva configuración a aplicar. Puesto que la comunicación se realiza a través de *Netopeer2*, el mensaje llega directamente a esta entidad (paso 2.1). Cuando lo ha recibido, realiza una conexión a *Sysrepo* mediante *socket unix* con la configuración recibida (paso 2.2). Esta entidad, consulta los diferentes modelos YANG frente a los que tiene que validar la configuración y los intenta validar (paso 2.3).

En el caso de pasar correctamente la validación, se realiza una conexión con el módulo que hemos diseñado a través de *socket unix* y le pasamos la configuración (paso 2.4). Dentro del procesamiento interno del propio módulo, se interpreta la información y se generan las estructuras internas con la información relevante (paso 2.5). Una vez creadas, se invoca a una función que hace uso de VICI para genere la información necesaria y la aplique directamente. Para ello, en primer lugar accede a las estructuras y usa la información necesaria para crear los datos que va a pasar

a IKE (paso 2.6), y finalmente aplica la configuración en caliente en el demonio de IKE (paso 2.7).

Podemos observar también, cuando IKE va a actualizar la información que posee, que se comunica directamente con la interfaz de VICI (paso 11.1). Cuando ha procesado la información, la envía directamente a *Sysrepo* haciendo uso de un *socket unix* (paso 11.2). Por último, envía los datos a notificar a *Netopeer2* y este envía la información directamente al controlador.

4.3. Módulo ietf-ipsec.py

A lo largo de esta sección, vamos a mostrar el funcionamiento y las estructuras creadas en el módulo (el código completo de este módulo y las estructuras comentadas está disponible en [40]). Este módulo estará siempre operativo dentro de cada dispositivo que actúe como nodo final en la topología SDN, en nuestro caso, en los dos nodos del escenario. Una vez iniciado, está a la escucha de nueva información recibida por el controlador, y cuando entran nuevos parámetros de configuración, obtiene la información necesario y crea diferentes estructuras en las que almacena la información recibida, de cara a tener la información más manejable y accesible. Genera tres estructuras por cada configuración nueva recibida, siendo estas SPD (*spd_entry*), PAD (*pad_entry*) e IKE (*ike_entry*).

4.3.1. Procesamiento de configuración recibida

Una vez iniciado el código y estando a la escucha de nuevas configuraciones, vamos a mostrar los pasos seguidos una vez recibe una configuración nueva hasta que ha sido aplicada en el *kernel* del dispositivo:

- 1. En primer lugar, nos suscribimos a los posibles cambios que se puedan recibir, en cuyo caso se ejecutará otra función que tratará la información. Podemos observar la suscripción a los nuevos cambios a continuación.

```
...
subscription = sr.Subscribe(session)
subscription.
    module_change_subscribe(module_name, module_change_cb, None)
...
```

- 2. Cuando recibimos una nueva configuración invocamos a una función (tal y como se comprueba en el fragmento anterior, se invoca a la función llamada *module_change_cb*), en la cual comprobamos el tipo de operación que se va a realizar (podría ser por ejemplo crear, modificar o eliminar, aunque en nuestro caso solo tratamos las configuraciones relacionadas con creación), donde será la opción de crear y aplicar una nueva configuración.
- 3. La información se va tratando por líneas llamando a una función concreta que se encargará de rellenar las estructuras para tener accesible la información (estas estructuras están inicializadas previamente), obteniendo de cada línea de entrada el atributo correspondiente, el valor que tiene y la entrada a la que pertenece. Comprobamos a que *database* pertenece esa línea, y después almacenamos el atributo y su valor correspondiente en su respectiva estructura, llamando a una función que tiene todos los posibles atributos para esa *database*. Podemos observar en el siguiente fragmento este paso y el anterior.

```

...
def print_change(op, old_val, new_val):
    if (op == sr.SR_OP_CREATED):
        ...
        if ("/IPsec/spd/spd-entry" in value):
            change_spd(value)

        elif ("/IPsec/pad/pad-entries" in value):
            change_pad(value)

        elif ("/ikev2/ike-connection/ike-conn-entries" in value):
            change_ike(value)
    ...

```

- 4. Una vez procesada toda la configuración entrante y actualizadas las estructuras, se invoca a una función que hace uso de la implementación de VICI, donde en primer lugar crea una estructura con la información de la conexión, y luego una segunda estructura relacionada con la clave compartida de IKE, instanciando ambas estructuras en el *kernel* directamente. Para ello usa las funciones *load-conn* y *load-shared*. En el código de la parte inferior podemos ver el contenido de la función nombrada, donde en primer lugar se crea un *socket* usado para crear una sesión de VICI. Acto seguido, hacemos uso de las estructuras rellenas previamente con los datos recibidos de configuración y, finalmente, creamos y cargamos estructuras correspondientes a conexión y secreto compartido con un formato comprensible por VICI. Esta información se carga en el demonio de IKE haciendo uso de la sesión creada.

```

...
def vici_charge_info():
    so = socket.socket(socket.AF_UNIX)
    so.connect("/var/run/charon.vici")
    s = vici.Session(so)
    connh1 = {"h2h":
        {"local_addrs": [ikeentry.local.ipv4],
         "remote_addrs": [ikeentry.remote.ipv4],
         "version": "ikeentry.version",
         "rekey_time": "ikeentry.rekey",
         "local":
             {"auth": "padentry.auth_method",
              "id": ikeentry.local.my_identifier},
         "remote":
             {"auth": "padentry.auth_method",
              "id": ikeentry.remote.my_identifier},
         "children":
             {"h2h":
                 {"start_action": "trap",
                  "rekey_time": "ikeentry.rekey",
                  "local_ts": [spdentry.condition.traffic_selector
                               _list.local_addresses.start],
                  "remote_ts": [spdentry.condition.traffic_selector

```

```

        _list.remote_addresses.start]]
    }
}
s.load_conn(connh1)
secreth1 = {"id": "ikekey",
            "type": "IKE",
            "data": padentry.auth_method.pre_shared.secret,
            "owners": [ikeentry.local.my_identifier]}
s.load_shared(secreth1)
...

```

- 5. Cuando la ejecución de la función termina, la configuración ya está cargada y funcionando en el dispositivo, y el módulo vuelve a estar a la escucha de nuevas configuraciones que aplicar.

4.3.2. Estructuras internas y atributos

Tal y como hemos visto en la sección anterior, el módulo crea estructuras que contienen información de SPD, PAD e IKE. En este apartado vamos a ir comentando para cada estructura, la información que se ha almacenado, en qué orden la hemos ido indicando y los valores que cada una de ellas puede tener. Para ello, mostraremos un resumen de los campos y ejemplos de valores, donde los campos en mayúscula son el nombre de un contenedor y dentro, cada atributo asociado a un valor de ejemplo.

```

SPD ENTRY INFO:
| Rule number -> 12
| NAMES
| | Name -> out/10.0.1.100/10.0.1.200
| CONDITION
| | TRAFFIC SELECTOR LIST
| | | Ts number -> 5
| | | Direction -> OUTBOUND
| | | LOCAL ADDRESSES
| | | | Start -> 10.0.1.100/32
| | | | End -> 10.0.1.100/32
| | | REMOTE ADDRESSES
| | | | Start -> 10.0.1.200/32
| | | | End -> 10.0.1.200/32
| | | Next layer protocol -> TCP
| | | LOCAL PORTS
| | | | Start -> 50
| | | | End -> 50
| | | REMOTE PORTS
| | | | Start -> 50
| | | | End -> 50
| PROCESSING INFO
| | Action -> PROTECT
| | IPsec SA CONF
| | | Security protocol -> esp

```

```
| | | Mode -> TRANSPORT
```

En el fragmento superior podemos observar los diferentes componentes que tendrá una entrada SPD. En primer lugar, una entrada posee un número de regla identificativo (*rule number*), junto a una lista de posibles nombres. Dentro de los atributos que tiene el tráfico, observamos principalmente la dirección que tendrá el tráfico, junto con un rango de direcciones que tendrá el dispositivo al que se aplica la entrada (*local addresses*) y el rango de direcciones que tendrá el dispositivo con el que se va a comunicar usando esta entrada concreta (*remote addresses*). Adicionalmente, también se indica de los puertos usados para la comunicación, distinguiendo al igual que las direcciones entre los diferentes dispositivos de la comunicación (*local ports* y *remote ports*).

En la zona inferior, se nos indican los diferentes parámetros relacionados con el procesamiento del tráfico (paquetes intercambiados en la comunicación los dispositivos involucrados). En ellos, indicamos que la acción a realizar sobre los paquetes es *PROTECT* (recordemos que era la opción de proteger el paquete, junto a dejarlo pasar y descartarlo), indicado en *action*. También podemos indicar el protocolo de seguridad a usar (en este caso concreto ESP, pudiéndose indicar también AH) junto al modo de transporte (en este caso es *TRANSPORT*, podría ser también *TUNNEL*), indicados en el fragmento en los atributos *security protocol* y *mode* respectivamente. Como comentaremos más en detalle en la siguiente sección, con respecto al protocolo, funcionará con ESP, donde se deja como posible mejora de este proyecto la posibilidad de funcionar también usando el protocolo AH. Por parte del modo de funcionamiento, podemos usar indistintamente tanto el modo túnel como el modo transporte (en caso de no indicar ningún modo de funcionamiento, por defecto funciona usando el modo túnel).

De manera similar, tendremos codificada la información relacionada con una entrada en PAD, mostrado un ejemplo en la parte inferior. Al principio, se indica un valor identificativo de la entrada en la tabla (*pad entry id*), junto con la dirección destino del tráfico de datos (*ipv4 address*). Se indica también que el protocolo de autenticación usado será IKEv2 en el atributo *pad auth protocol* (recordamos que es el protocolo en el que delega IPsec para la negociación de claves). Dentro del contenedor del método de autenticación, se indica, en primer lugar, el método de autenticación (en nuestro caso solo haremos uso de clave compartida previamente, *pre-shared*), y posteriormente el valor que tendrá el secreto compartido, que están indicados en los atributos *auth m* y *secret* respectivamente.

```
PAD ENTRY INFO:
| Pad entry id -> 15
| Ipv4 address -> 10.0.1.200
| Pad auth protocol -> IKEv2
| AUTH METHOD
```

```
| | Auth m -> pre-shared  
| | PRE SHARED  
| | | Secret -> secreto_compartido
```

```
IKE ENTRY INFO:  
| Connection name -> testconn  
| Autostartup -> ALWAYSON  
| Version -> ikev2  
| Ike reauth lifetime -> 120  
| Ike sa lifetime -> 120  
| IPsec sa lifetime -> 60  
| DH group -> 2048  
| LOCAL  
| | Ipv4 -> 10.0.1.100  
| | My identifier -> User1  
| REMOTE  
| | Ipv4 -> 10.0.1.200  
| | My identifier -> User2  
| Pfs group -> 0
```

Por último, podemos observar en el fragmento superior los diferentes atributos de una entrada de IKE. En ella, podemos destacar el nombre de la conexión (*connection name*), la versión (a lo largo de la prueba usaremos la versión 2, indicado en el atributo *version*) y los tiempos de expiración de las diferentes claves, el cual una vez concluido requiere volver a negociar las claves (diferentes campos *lifetime*). Se indica también para ambos dispositivos involucrados en la comunicación tanto la dirección concreta como un identificador (contenido de las secciones *local* y *remote*). A lo largo del desarrollo de la solución probaremos el uso de diferentes valores en los tiempos de expiración.

5. Despliegue del escenario y pruebas

A lo largo de esta sección, se van a mostrar el despliegue y la realización de las pruebas de concepto que vamos a comentar. Toda la prueba ha sido realizada bajo un escenario totalmente virtualizado, simulando los diferentes componentes del escenario. Mostraremos también diferentes atributos dentro de las configuraciones, como por ejemplo la posibilidad de funcionamiento tanto en modo túnel como transporte correspondiente.

5.1. Despliegue del escenario

En primer lugar, antes de realizar la prueba es necesario un despliegue del escenario. Puesto que el despliegue físico requiere de una gran cantidad de recursos y la virtualización nos proporciona sencillez y flexibilidad, el escenario que vamos a desplegar es totalmente virtual, y va a estar compuesto de un controlador y dos nodos de la red a gestionar, es decir, un escenario *host-to-host* (seleccionado de entre las diferentes opciones que se presentan en la sección 2.7.2). Sobre este escenario, se probarán diferentes configuraciones y se comprobará que se aplican correctamente a los nodos. El escenario, junto a las direcciones asociadas a cada interfaz de cada elemento se observan en la Figura 20.

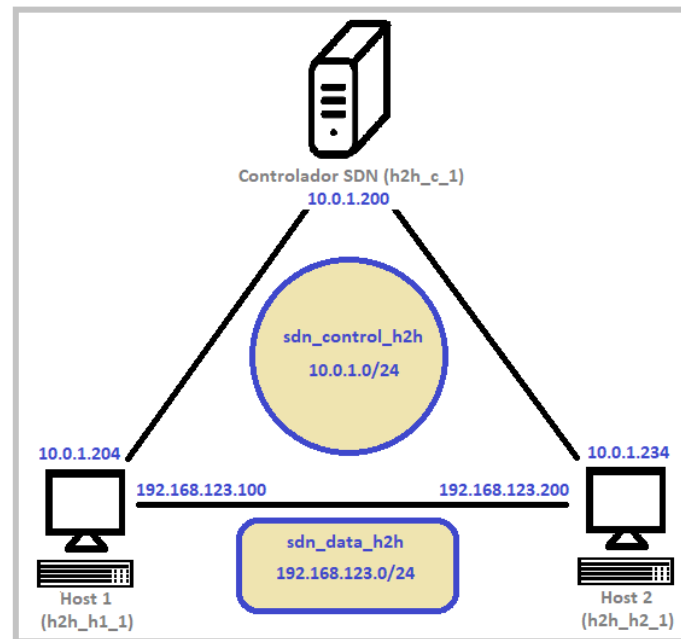


Figura 20: Topología final del escenario del caso de uso.

Para el lanzamiento del escenario virtualizado, se hace uso de *docker* para la

creación de los tres contenedores correspondientes. El código necesario para arrancar el escenario con *docker* se muestra en su totalidad en el Anexo A.

Una vez iniciado *docker*, tendremos tres contenedores corriendo (simulando cada uno un elemento de nuestra prueba de concepto) y dos redes simuladas, una entre los nodos y otra entre el controlador y los nodos. Podemos observar en la Figura 20 los diferentes dispositivos que componen la topología, que son los simulados por los contenedores, donde tenemos en primer lugar el controlador del escenario (llamado *sdn_control_h2h*) y en la parte inferior los dos *hosts* que se van a configurar (llamados *h2h_h1_1* y *h2h_h2_1*). Disponemos en primer lugar de una red que comunica el controlador con los diferentes nodos a configurar, llamada *sdn_control_h2h* y con direccionamiento *10.0.1.0/24*). En esta red estará funcionando NETCONF para simplificar la conexión y la aplicación de asociaciones de seguridad. Por parte de la red que une a los diferentes *hosts*, llamada *sdn_data_h2h*, posee direccionamiento *192.168.123.0/24*. En la figura se pueden observar los direccionamientos aplicados a cada interfaz del escenario.

En cada uno de los nodos estará implementado y configurado NETCONF, y por tanto *Netopeer*, permitiéndonos la instanciación y la gestión de las asociaciones de seguridad entre los diferentes nodos y ejecutando el cliente en el controlador y los servidores en los dispositivos a configurar. Por parte del cliente (*Netopeer2-cli*), nos permite conectarnos con una instancia de servidor usando NETCONF e iniciar una sesión. Mediante esa sesión, se puede configurar la información del servidor. En nuestra prueba de concepto, esta interfaz estará implementada en el controlador, puesto que juega el papel de entidad central encargada de gestionar al resto de dispositivos. Los comandos utilizados serán principalmente *connect*, *subscribe*, *edit-config*, *get-config* y *disconnect*. Recordemos que la comunicación se realizaba con mensajes RCP, por lo tanto, la interfaz tiene que ser capaz de generar y enviar este tipo de mensajes.

Por parte del servidor, nos permite la recepción de las diferentes directrices por parte del cliente, tras la creación de sesiones usando NETCONF. En nuestra prueba de concepto, el servidor será implementado en los diferentes nodos a configurar por el controlador. El dispositivo que implemente esta interfaz debe disponer de un módulo YANG frente al que validar los distintos mensajes intercambiados. También debe ser capaz de recibir e interpretar los diferentes mensajes RCP. Como está implementado en el dispositivo que se va a configurar, la interfaz debe ser capaz también de poder modificar las diferentes *datastores* en función la información recibida por parte del controlador.

5.2. Prueba final de funcionamiento

Una vez presentada toda la información necesaria, vamos a realizar la prueba final de concepto para mostrar al lector el correcto funcionamiento de todo lo expuesto en el proyecto. Vamos a ver también como conseguimos el objetivo marcado, donde instanciamos asociaciones de seguridad y el propio nodo las aplica *en caliente* sin necesidad de ficheros ni reiniciar ningún servicio. Para ello, vamos a desplegar el escenario mostrado en el apartado anterior, instanciando una nueva configuración en los nodos, y comprobando que la comunicación se realiza de forma segura siguiendo las directrices que marca la nueva configuración. La realización de la prueba la haremos mostrando los dos modos fundamentales de funcionamiento que tiene IPsec, tanto modo transporte como modo túnel.

Importante destacar que se presentan en el Anexo B los archivos XML que han sido utilizados en el desarrollo de las pruebas para instanciar las políticas en cada uno de los *hosts* desde el controlador. Por último, tras haber expuesto el correcto funcionamiento de las pruebas, mostraremos tanto el contenido de las estructuras como el contenido de la llamada a VICI dentro del módulo creado.

5.2.1. Prueba de funcionamiento en modo transporte

En primer lugar, lanzamos los contenedores con los tres elementos de la topología simulados. Una vez operativos, en primer lugar en los nodos lanzamos el módulo *ietf-ipsec.py* para que los nodos queden a la escucha de nuevas configuraciones. En el lado del controlador, usamos *netopeer2-cli* para conectarnos con cada uno de los nodos y cargar la correspondiente configuración. Los pasos que seguimos para la instanciación de las políticas en el controlador son los siguientes:

```
# netopeer2-cli

> connect --host 10.0.1.204 --ssh --login netconf
> subscribe
> edit-config --target running --config=/opt/dev/cfgIPsec2/client-xm1s
  /case1/h2h_transport_esp_enc_auth_h1_spd_pad_ike.xml
> get-config --source=running
> disconnect

> connect --host 10.0.1.234 --ssh --login netconf
> subscribe
> edit-config --target running --config=/opt/dev/cfgIPsec2/client-xm1s
  /case1/h2h_transport_esp_enc_auth_h2_spd_pad_ike.xml
> get-config --source=running
> disconnect
```

Una vez dentro de *netopeer2-cli*, nos conectamos con el *host 1* de nuestra topología (cuya dirección es 10.0.1.204) mediante SSH. Nos suscribimos a los cambios

y, acto seguido, le enviamos la configuración creada para la asociación de seguridad entre este y el *host 2*. Una vez ha sido aplicada, podemos obtener la configuración para comprobar que se ha actualizado correctamente, y finalmente cerramos la sesión. Para el *host 2* se procede de manera similar, donde los únicos cambios son la dirección (en este caso la dirección es 10.0.1.234) y el archivo que contiene la configuración necesaria para la asociación de seguridad.

En ambos casos, en el momento que se usa el comando *edit-config* con la información que se quiere añadir, en el nodo correspondiente el módulo recibe la información y la trata correctamente, instanciando directamente los cambios en el *kernel*. Una vez están aplicadas las configuraciones, vamos a comprobar que realmente están ya instanciadas en el *kernel*. Para ello, en primer lugar en el *host 1*, vamos a comprobar la correcta instanciación comprobando las salidas tanto de *ip xfrm policy* como de *swanctl -L*, observando la salida en la Figura 21.

```
root@h1:/home/netconf/cfgipsec2# swanctl -L
h2h: IKEv2, no reauthentication, rekeying every 3600s
  local: 192.168.123.100
  remote: 192.168.123.200
  local pre-shared key authentication:
    id: Ignacio
  remote pre-shared key authentication:
    id: Ruben
h2h: TRANSPORT, rekeying every 7200s
  local: 192.168.123.100/32
  remote: 192.168.123.200/32
root@h1:/home/netconf/cfgipsec2# ip xfrm policy
src 192.168.123.100/32 dst 192.168.123.200/32
  dir out priority 367231
    tmpl src 0.0.0.0 dst 0.0.0.0
      proto esp spi 0xc9c469b4 reqid 1 mode transport
src 192.168.123.200/32 dst 192.168.123.100/32
  dir in priority 367231
    tmpl src 0.0.0.0 dst 0.0.0.0
      proto esp reqid 1 mode transport
```

Figura 21: Información en *host 1* una vez aplicada la configuración (transporte).

Podemos observar en la configuración, que se han aplicado todos los parámetros correctamente, comenzando por las direcciones IP y los propios nombres de los usuarios. Vemos también reflejado el uso del modo transporte, y los tiempos de expiración correctos que hemos indicado en el XML instanciado desde el controlador. De igual manera, por parte del *host 2*, una vez instanciado su respectivo archivo XML con la configuración, usamos los mismos comandos para comprobar que la configuración se ha actualizado correctamente, y vemos que así ha sucedido en la Figura 22.

Una vez aplicada la configuración, el tráfico intercambiado entre los dos nodos que hemos configurado se realizará de forma segura. De esta manera, cogemos el *host 1* y procedemos a generar tráfico en dirección al *host 2* (podría haber sido indistintamente en dirección contraria). En el momento que se comienza a generar el tráfico, se encuentra una entrada en la PAD y se comienza una negociación IKE con

```

root@h2:/home/netconf/cfgipsec2# swanctl -L
h2h: IKEv2, no reauthentication, rekeying every 3600s
local: 192.168.123.200
remote: 192.168.123.100
local pre-shared key authentication:
id: Ruben
remote pre-shared key authentication:
id: Ignacio
h2h: TRANSPORT, rekeying every 7200s
local: 192.168.123.200/32
remote: 192.168.123.100/32
root@h2:/home/netconf/cfgipsec2# ip xfrm policy
src 192.168.123.200/32 dst 192.168.123.100/32
dir out priority 367231
tmpl src 0.0.0.0 dst 0.0.0.0
proto esp spi 0xcf81a519 reqid 1 mode transport
src 192.168.123.100/32 dst 192.168.123.200/32
dir in priority 367231
tmpl src 0.0.0.0 dst 0.0.0.0
proto esp reqid 1 mode transport

```

Figura 22: Información en *host 2* una vez aplicada la configuración (transporte).

3066	198.316801779	192.168.123.100	192.168.123.200	ISAKMP	936	IKE_SA_INIT MID=00 Initiator Request
3067	198.319540434	192.168.123.200	192.168.123.100	ISAKMP	312	IKE_SA_INIT MID=00 Responder Response
3068	198.321474319	192.168.123.100	192.168.123.200	ISAKMP	414	IKE_AUTH MID=01 Initiator Request
3069	198.361485629	192.168.123.200	192.168.123.100	ISAKMP	270	IKE_AUTH MID=01 Responder Response
3070	199.328119904	192.168.123.100	192.168.123.200	ESP	154	ESP (SPI=0xc9c469b4)
3071	199.328261005	192.168.123.200	192.168.123.100	ESP	154	ESP (SPI=0xcf81a519)
3072	200.352372114	192.168.123.100	192.168.123.200	ESP	154	ESP (SPI=0xc9c469b4)
3073	200.352485758	192.168.123.200	192.168.123.100	ESP	154	ESP (SPI=0xcf81a519)

Figura 23: Negociación IKE y paquetes ICMP protegidos en modo transporte.

el otro dispositivo. Se suceden los distintos mensajes sobre los que hablamos más en detalle en la sección 2.4.2, de tal manera que se negocian las claves y se inicia una asociación de seguridad entre ellos.

Una vez terminada la negociación, empiezan a intercambiar tráfico, que ya está securizado. Podemos observar en la Figura 23 tanto la negociación de IKE como el intercambio de tráfico seguro tras la negociación. Podemos observar tanto en la negociación IKE como en el tráfico ya protegido como el *host 1* es el encargado de iniciar la comunicación. En este caso, hemos usado ICMP para realizar *ping* a modo de generación de tráfico.

5.2.2. Prueba de funcionamiento en modo túnel

Para la realización de la prueba en modo túnel, una vez lanzados los contenedores y los módulos operativos, usamos el controlador para instanciar las configuraciones en los dispositivos de la topología. Para ello, hacemos uso de nuevo de *netopeer2-cli* con la idea de conectarnos a los dispositivos y configurarlos. Los pasos, son similares al caso anterior, cambiando solamente el archivo de configuración, donde indicaremos esta vez el funcionamiento en modo túnel (realmente omitimos el atributo del modo, ya que por defecto se trabaja en modo túnel), observados a continuación:

```
# netopeer2-cli

> connect --host 10.0.1.204 --ssh --login netconf
> subscribe
> edit-config --target running --config=/opt/dev/cfgIPsec2/client-xm1s
  /case1/h2h_tunnel_esp_enc_auth_h1_spd_pad_ike.xml
> get-config --source=running
> disconnect

> connect --host 10.0.1.234 --ssh --login netconf
> subscribe
> edit-config --target running --config=/opt/dev/cfgIPsec2/client-xm1s
  /case1/h2h_tunnel_esp_enc_auth_h2_spd_pad_ike.xml
> get-config --source=running
> disconnect
```

Procediendo de manera similar al caso anterior, nos conectamos a ambos dispositivos de manera secuencial, suscribiéndonos a los cambios que reciban y cargando la respectiva configuración en cada uno. Una vez hemos aplicado los diferentes cambios en ambos dispositivos, estos reciben la configuración, la procesan y la instancian directamente en el *kernel*. Cuando está la información cargada, ambos dispositivos pueden realizar la negociación IKE y comenzar a intercambiar tráfico securizado.

```
root@h1:/home/netconf/cfgipsec2# swanctl -L
h2h: IKEv2, no reauthentication, rekeying every 3600s
  local: 192.168.123.100
  remote: 192.168.123.200
  local pre-shared key authentication:
    id: Ignacio
  remote pre-shared key authentication:
    id: Ruben
h2h: TUNNEL, rekeying every 7200s
  local: 192.168.123.100/32
  remote: 192.168.123.200/32
root@h1:/home/netconf/cfgipsec2# ip xfrm policy
src 192.168.123.100/32 dst 192.168.123.200/32
  dir out priority 367232
    tmpl src 192.168.123.100 dst 192.168.123.200
      proto esp reqid 1 mode tunnel
src 192.168.123.200/32 dst 192.168.123.100/32
  dir fwd priority 367232
    tmpl src 192.168.123.200 dst 192.168.123.100
      proto esp reqid 1 mode tunnel
src 192.168.123.200/32 dst 192.168.123.100/32
  dir in priority 367232
    tmpl src 192.168.123.200 dst 192.168.123.100
      proto esp reqid 1 mode tunnel
```

Figura 24: Información en *host 1* una vez aplicada la configuración (túnel).

```

root@h2:/home/netconf/cfgipsec2# swanctl -L
h2h: IKEv2, no reauthentication, rekeying every 3600s
  local: 192.168.123.200
  remote: 192.168.123.100
  local pre-shared key authentication:
    id: Ruben
  remote pre-shared key authentication:
    id: Ignacio
h2h: TUNNEL, rekeying every 7200s
  local: 192.168.123.200/32
  remote: 192.168.123.100/32
root@h2:/home/netconf/cfgipsec2# ip xfrm policy
src 192.168.123.200/32 dst 192.168.123.100/32
  dir out priority 367232
  tmpl src 192.168.123.200 dst 192.168.123.100
    proto esp reqid 1 mode tunnel
src 192.168.123.100/32 dst 192.168.123.200/32
  dir fwd priority 367232
  tmpl src 192.168.123.100 dst 192.168.123.200
    proto esp reqid 1 mode tunnel
src 192.168.123.100/32 dst 192.168.123.200/32
  dir in priority 367232
  tmpl src 192.168.123.100 dst 192.168.123.200
    proto esp reqid 1 mode tunnel

```

Figura 25: Información en *host 1* una vez aplicada la configuración (túnel).

Podemos observar que la configuración se ha cargado correctamente en ambos dispositivos, tanto en la Figura 24 como en la Figura 25. En la primera figura observamos, por parte del *host 1*, que se ha aplicado la configuración correctamente, con la única diferencia de que la comunicación se realizará en modo túnel. Por parte del *host 2*, podemos observar que la única diferencia con respecto a la prueba anterior radica también en la configuración del modo, ya que en este caso la comunicación se realizará en modo túnel. En ambas configuraciones podemos ver una tercera entrada dentro de la ejecución del comando *ip xfrm policy*, entrada que indica la necesidad de encapsular los paquetes que se dirigen a la dirección *192.168.123.200* (perteneciente al *host 2*), quedando retratado el funcionamiento del modo túnel con respecto a la configuración de la prueba de funcionamiento anterior.

5	5.610829677	192.168.123.100	192.168.123.200	ISAKMP	936	IKE_SA_INIT MID=00 Initiator Request
6	5.611590695	192.168.123.200	192.168.123.100	ISAKMP	312	IKE_SA_INIT MID=00 Responder Response
7	5.612211404	192.168.123.100	192.168.123.200	ISAKMP	398	IKE_AUTH MID=01 Initiator Request
8	5.632060411	192.168.123.200	192.168.123.100	ISAKMP	270	IKE_AUTH MID=01 Responder Response
9	6.610726338	192.168.123.100	192.168.123.200	ESP	170	ESP (SPI=0xcc3b9fdb)
10	6.610857841	192.168.123.200	192.168.123.100	ESP	170	ESP (SPI=0xcd06ccbd)
11	7.634731080	192.168.123.100	192.168.123.200	ESP	170	ESP (SPI=0xcc3b9fdb)
12	7.634844797	192.168.123.200	192.168.123.100	ESP	170	ESP (SPI=0xcd06ccbd)

Figura 26: Negociación IKE y paquetes ICMP protegidos en modo túnel.

Una vez comprobado que la configuración ha sido instanciada correctamente, generamos tráfico entre los dos dispositivos. Podemos observar en la Figura 26 como, una vez se genera tráfico se suceden los mensajes iniciales de la negociación de IKE hasta la generación de la asociación de seguridad. Una vez ha terminado la negociación, se sucede el tráfico generado de manera segura entre los dispositivos.

5.2.3. Contenido de estructuras internas

Por último, vamos a mostrar al lector como quedan actualizadas las tablas que creamos en el módulo *ietf-ipsec.py*, una vez recibido y procesado el archivo XML con la configuración a aplicar. Importante recordar que se disponía de tres estructuras, para guardar información relacionada con SPD, PAD e IKE, y que los ficheros XML usados se encuentran reflejados en el Anexo B de la documentación. Por último, indicar que mostramos el contenido final en el módulo que se está ejecutando en el *host 1*, tras realizar la primera prueba descrita anteriormente.

En primer lugar, mostramos el estado de la estructura relacionada con SPD. En ella, podemos destacar las direcciones que poseen ambos dispositivos finales, que son efectivamente *192.168.123.100* y *192.168.123.200*. Notable también la indicación de que se va a proteger el tráfico, que se va a usar el protocolo ESP y que IPsec va a funcionar en modo transporte.

```
SPD ENTRY INFO:
| Rule number -> 1
| NAMES
| | Name -> out/192.168.123.100/192.168.123.200
| CONDITION
| | TRAFFIC SELECTOR LIST
| | | Ts number -> 7
| | | Direction -> OUTBOUND
| | | LOCAL ADDRESSES
| | | | Start -> 192.168.123.100/32
| | | | End -> 192.168.123.100/32
| | | REMOTE ADDRESSES
| | | | Start -> 192.168.123.200/32
| | | | End -> 192.168.123.200/32
| | | Next layer protocol -> TCP
| | | LOCAL PORTS
| | | | Start -> 0
| | | | End -> 0
| | | REMOTE PORTS
| | | | Start -> 0
| | | | End -> 0
| PROCESSING INFO
| | Action -> PROTECT
| | IPsec SA CONF
| | | Security protocol -> esp
| | | Mode -> TRANSPORT
```

A continuación observamos el contenido de la estructura que posee la información de PAD. En ella, podemos destacar principalmente el uso del protocolo de autenticación IKEv2 y el método de autenticación basado en clave compartida previamente, mostrando el valor del propio secreto.

```
PAD ENTRY INFO:
| Pad entry id -> 7
| Ipv4 address -> 192.168.123.200
| Pad auth protocol -> IKEv2
| AUTH METHOD
| | Auth m -> pre-shared
| | PRE SHARED
| | | Secret -> secreto_compartido
```

Por último, tenemos indicado en la parte inferior la información relacionada con la estructura de IKE. En ella, podemos observar principalmente los identificadores y las direcciones IP de los dos dispositivos involucrados en la asociación de seguridad.

```
IKE ENTRY INFO:
| Connection name -> gateway2
| Autostartup -> ALWAYSON
| Version -> ikev2
| Ike reauth lifetime -> 0
| Ike sa lifetime -> 0
| IPsec sa lifetime -> 0
| DH group -> 2048
| LOCAL
| | Ipv4 -> 192.168.123.100
| | My identifier -> Ignacio
| REMOTE
| | Ipv4 -> 192.168.123.200
| | My identifier -> Ruben
| Pfs group -> 0
```


6. Conclusiones y vías futuras

A lo largo del desarrollo del proyecto se ha indagado en los escenarios SDN, y más concretamente en la gestión de asociaciones de seguridad IPsec entre diferentes nodos de la red. En estos escenarios, el controlador gestiona y configura los recursos de red distribuidos y proporciona una vista abstracta de los recursos de red a las aplicaciones SDN. Esta mejora permite una gran cantidad de ventajas a las empresas relacionadas con estos tipos de servicios, ofreciendo una mejora tanto en calidad de servicio como en tiempo de respuesta, derivando tanto en una mejor experiencia de usuario como un beneficio económico para las diferentes empresas relacionadas con el sector.

Tal y como hemos ido comentando, la gestión de la seguridad en este tipo de escenarios es bastante pobre. Sumado al constante crecimiento de dispositivos y la alta necesidad de ofrecer un sistema seguro, el desarrollo de este trabajo se centró en mejorar la gestión de las configuraciones de seguridad. La realización no ha partido desde cero, puesto que disponemos de material que han trabajado otros compañeros sobre el que hemos trabajado nosotros, seleccionando algunas ramas de investigación de las propuestas en estos documentos. En concreto, nos hemos centrado en la solución planteada en [1], donde se nos plantea la gestión automatizada de las asociaciones de IPsec a través de un controlador SDN. El módulo encargado de tratar la configuración en los dispositivos de red ya se encuentra implementado en C. Era necesaria una implementación en un lenguaje que nos permitiera una velocidad mayor a la hora de realizar pruebas y de configurarlos, por lo que se seleccionó *Python* para el desarrollo del módulo, llamado *ietf-ipsec.py*.

La adición de este módulo es mejora la más extensa, puesto que al poder instanciar la configuración *en caliente* no se necesita el uso de ficheros externos de configuración ni estar reiniciando ningún servicio, y se tiene todo el proceso automatizado. En la implementación del módulo, hemos hecho uso de una librería de *StrongSwan* que implementa VICI, y hemos desarrollado una función que nos permite instanciar una configuración en el momento que se recibe. Para procesar la información, hemos creado unas estructuras en las que almacenamos los datos relevantes de la configuración recibida (estas estructuras serán consultadas por VICI para instanciar la configuración). En resumen, se consigue una automatización en la aplicación de las configuraciones, aplicando los cambios en caliente en el *kernel* del dispositivo. El código del módulo y las estructuras están accesibles en [40].

El funcionamiento del módulo se ha comprobado a través de una sencilla prueba de concepto que hemos descrito previamente en la documentación, sobre la que instanciamos desde el controlador configuración a dos dispositivos para que se comuniquen de forma segura con IPsec, mostrando como se aplican las políticas y se comunican de forma segura a partir de ese momento. La prueba se realiza con un

número muy limitado de dispositivos, lo que no quita que este escenario pueda usarse en topologías mucho más grandes, quedando como mejora futura la implantación en escenarios más complejos.

En cuanto a las líneas de trabajo futuras, relacionadas con este trabajo de fin de grado, queda abierta la posibilidad de implementar en el módulo el envío de notificaciones al controlador cuando suceda un evento o el envío del estado de las asociaciones en el dispositivo en un momento determinado. También serían una buena línea de investigación la posibilidad de eliminar asociaciones de seguridad que han sido ya instanciadas en el dispositivo o la posibilidad de cargar configuraciones que hagan uso del protocolo AH. En resumen, las vías de investigación y desarrollo se centran en completar la implementación del módulo que se tenía en C, pero en *Python*.

Referencias

- [1] *Draft* relacionado con IPsec, en el que se explican en más detalle los ejemplos comentados. <https://datatracker.ietf.org/doc/html/draft-ietf-i2nsf-sdn-IPsec-flow-protection-08>
- [2] Estándar SDN en IETF. <https://tools.ietf.org/html/rfc7426>
- [3] Estándar SNMP en IETF. <https://tools.ietf.org/html/rfc3413>
- [4] Estándar NETCONF en IETF. <https://tools.ietf.org/html/rfc6241>
- [5] Estándar RPC en IETF. <https://tools.ietf.org/html/rfc5531>
- [6] Estándar XML en IETF. <https://tools.ietf.org/html/rfc4826>
- [7] Estándar YANG en IETF. <https://tools.ietf.org/html/rfc7950>
- [8] Estándar SSH en IETF. <https://tools.ietf.org/html/rfc4253>
- [9] Estándar TLS en IETF. <https://tools.ietf.org/html/rfc8446>
- [10] Ilustración sobre la arquitectura de NETCONF obtenida en <https://es.wikipedia.org/wiki/NETCONF>.
- [11] Estándar IPsec en IETF. <https://tools.ietf.org/html/rfc6071>
- [12] Estándar AH en IETF. <https://tools.ietf.org/html/rfc4302>
- [13] Estándar MD5 en IETF. <https://tools.ietf.org/html/rfc1321>
- [14] Estándar SHA1 en IETF. <https://tools.ietf.org/html/rfc3174>
- [15] Estándar ESP en IETF. <https://tools.ietf.org/html/rfc4303>
- [16] Estándar 3DES en IETF. <https://tools.ietf.org/html/rfc1851>
- [17] Estándar AES en IETF. <https://tools.ietf.org/html/rfc3826>
- [18] Información sobre SPI en <https://www.ibm.com/support/pages/what-security-parameter-index-spi>.
- [19] Imagen obtenida de <https://docs.oracle.com/cd/E19683-01/817-2694/IPsec-ov-1/index.html>.
- [20] Imagen obtenida de <https://security.stackexchange.com/questions/145708/in-IPsec-what-is-the-definition-purpose-of-sad-and-spd>.
- [21] Estándar relacionado con asociaciones de seguridad y su arquitectura. <https://tools.ietf.org/html/rfc4301>

- [22] Imagen obtenida de <https://lira.epac.to/DOCS-TECH/Security/Internet%20Security%20Technology%20and%20Hacking/final/ch15lev1sec2.html>.
- [23] Estándar IKE en IETF. <https://tools.ietf.org/html/rfc2409>
- [24] Estándar Diffie-Hellman en IETF. <https://tools.ietf.org/html/rfc2631>
- [25] Estándar IKEv2 en IETF. <https://tools.ietf.org/html/rfc5996>
- [26] Página oficial relacionada con OpenSwan en <https://www.openswan.org/>
- [27] Página oficial de LibreSwan en <https://libreswan.org/>
- [28] Página oficial de StrongSwan en <https://www.strongswan.org>.
- [29] Información sobre el demonio *charon* en <https://wiki.strongswan.org/projects/strongswan/wiki/Charon>.
- [30] Información relacionada con VICI en <https://wiki.strongswan.org/projects/strongswan/wiki/Vici>.
- [31] Información sobre Swantcl en la página <https://wiki.strongswan.org/projects/strongswan/wiki/swantcl>.
- [32] Información detallada sobre el funcionamiento interno de VICI. https://www.strongswan.org/apidoc/md_src_libcharon_plugins_vici_README.html
- [33] Trabajo de Fin de Grado desarrollado por Ignacio Martínez Alpiste, titulado *Configuración centralizada de IKE en múltiples dispositivos con el uso de SDNs*.
- [34] *Draft* relacionado con la protección IPsec, creado por Rafael Marín López & Gabriel López Millán, ubicado en <https://tools.ietf.org/html/draft-ietf-i2nsf-sdn-IPsec-flow-protection-00>
- [35] *Gitlab* relacionado con el desarrollo del trabajo, donde se muestran los tres principales escenarios y como ponerlos en funcionamiento, ubicado en <https://gitlab.atika.um.es/gabilm.um.es/sysrepo-netopeer2-cfgIPsec2/>
- [36] Página principal de *Sysrepo* en <http://www.sysrepo.org/>
- [37] Información sobre *Netopeer* en <https://github.com/CESNET/netopeer>
- [38] Información relacionada con *libyang* en <https://awesomeopensource.com/project/CESNET/libyang>
- [39] Información sobre la segunda versión de *Netopeer* en <https://github.com/CESNET/Netopeer2>
- [40] Código del módulo `ietf-ipsec.py` y estructuras diseñadas en este proyecto. <https://gitlab.atika.um.es/gabilm.um.es/cfgIPsec3/tree/IKE>

Anexo A - Puesta en marcha del escenario

En este primer anexo, vamos a mostrar los diferentes ficheros usados para lanzar el escenario virtualizado y tenerlo totalmente operativo. En primer lugar vamos a mostrar el fichero *docker-compose.yml*, en el que definimos los diferentes dispositivos y redes de nuestro escenario.

```
version: '3.2'
services:
  c:
    build: base/netopeer-cli
    hostname: c
    cap_add:
      - SYS_ADMIN
      - NET_ADMIN
    networks:
      sdn_control_h2h:
        ipv4_address: 10.0.1.200
    volumes:
      - type: bind
        source: ./filesc
        target: /home/netconf/cfgIPsec2/filesc
  h1:
    build: base/netopeer-server
    hostname: h1
    cap_add:
      - SYS_ADMIN
      - NET_ADMIN
    environment:
      - NETCONF_USER=netconf
      - NETCONF_PASSWORD=netconf
      - SDN_NET=10.0.1.0/24
    depends_on:
      - "c"
    networks:
      sdn_control_h2h:
        ipv4_address: 10.0.1.204
      sdn_data_h2h:
        ipv4_address: 192.168.123.100
    volumes:
      - type: bind
        source: ./filesh1
        target: /home/netconf/cfgIPsec2/filesh1
  h2:
    hostname: h2
    build: base/netopeer-server
    cap_add:
      - SYS_ADMIN
      - NET_ADMIN
    environment:
      - NETCONF_USER=netconf
      - NETCONF_PASSWORD=netconf
```

```

- SDN_NET=10.0.1.0/24
depends_on:
- "c"
networks:
  sdn_control_h2h:
    ipv4_address: 10.0.1.234
  sdn_data_h2h:
    ipv4_address: 192.168.123.200
volumes:
- type: bind
  source: ./files2
  target: /home/netconf/cfgIPsec2/files2
networks:
  sdn_control_h2h:
    driver: bridge
    ipam:
      config:
        - subnet: 10.0.1.0/24
    driver_opts:
      com.docker.network.bridge.name: sdn_control_h2h
  sdn_data_h2h:
    driver: bridge
    ipam:
      config:
        - subnet: 192.168.123.0/24
    driver_opts:
      com.docker.network.bridge.name: sdn_data_h2h

```

Podemos observar en el contenido mostrado anteriormente que se hace uso del archivo *netopeer-cli* para iniciar al controlador y del archivo *netopeer-server* para iniciar los dispositivos finales. Ha sido necesario el desarrollo de distintas versiones hasta conseguir una versión estable de estos ficheros, y vamos a mostrarlos a continuación, empezando por el contenido de *netopeer-cli*.

```

FROM ubuntu:18.04

MAINTAINER mislav.novakovic@sartura.hr

RUN \
  apt-get update && apt-get install -y \
  # general tools
  net-tools \
  git \
  cmake \
  build-essential \
  vim \
  supervisor \
  # libyang
  libpcap-dev \
  pkg-config \
  # sysrepo

```

```
libavl-dev \  
libev-dev \  
libprotobuf-c-dev \  
protobuf-c-compiler \  
# netopeer2 \  
libssh-dev \  
libssl-dev \  
# bindings  
swig \  
python3-dev \  
python3-pip  
  
# add netconf user  
RUN \  
    adduser --system netconf && \  
    echo "netconf:netconf" | chpasswd  
  
# generate ssh keys for netconf user  
RUN \  
    mkdir -p /home/netconf/.ssh && \  
    ssh-keygen -A && \  
    ssh-keygen -t dsa -P '' -f /home/netconf/.ssh/id_dsa && \  
    cat /home/netconf/.ssh/id_dsa.pub > /home/netconf/.ssh/  
authorized_keys  
  
# use /opt/dev as working directory  
RUN mkdir /opt/dev  
WORKDIR /opt/dev  
  
# libyang  
RUN \  
    git clone https://github.com/CESNET/libyang.git && \  
    cd libyang && mkdir build && cd build && \  
    git checkout devel && \  
    cmake -DCMAKE_INSTALL_PREFIX:PATH=/usr -DGEN_LANGUAGE_BINDINGS=  
ON -DCMAKE_BUILD_TYPE:String="Debug" -DENABLE_BUILD_TESTS=OFF ..  
&& \  
    make -j2 && \  
    make install && \  
    ldconfig  
  
# sysrepo  
RUN \  
    git clone https://github.com/sysrepo/sysrepo.git && \  
    cd sysrepo && mkdir build && cd build && \  
    git checkout devel && \  
    cmake -DCMAKE_INSTALL_PREFIX:PATH=/usr -DGEN_LANGUAGE_BINDINGS=  
ON -DCMAKE_BUILD_TYPE:String="Debug" -DENABLE_TESTS=OFF -  
DREPOSITORY_LOC:PATH=/etc/sysrepo .. && \  
    make -j2 && \  
    make install && \  
    ldconfig
```

```

# libnetconf2
RUN \
    git clone https://github.com/CESNET/libnetconf2.git && \
    cd libnetconf2 && mkdir build && cd build && \
    git checkout devel && \
    cmake -DCMAKE_INSTALL_PREFIX:PATH=/usr -DGEN_LANGUAGE_BINDINGS=
ON -DCMAKE_BUILD_TYPE:String="Debug" -DENABLE_BUILD_TESTS=OFF ..
&& \
    make -j2 && \
    make install && \
    ldconfig

# keystore
#RUN \
#    cd /opt/dev && \
#    git clone https://github.com/CESNET/Netopeer2.git && \
#    cd Netopeer2 && \
#    cd keystore && mkdir build && cd build && \
#    git checkout devel-server && \
#    cmake -DCMAKE_INSTALL_PREFIX:PATH=/usr -DGEN_LANGUAGE_BINDINGS=
ON -DCMAKE_BUILD_TYPE:String="Debug" .. && \
#    make -j2 && \
#    make install && \
#    ldconfig

# netopeer2
RUN \
    cd /opt/dev && \
    git clone https://github.com/CESNET/Netopeer2.git && \
    cd Netopeer2/server && mkdir build && cd build && \
    cmake -DCMAKE_INSTALL_PREFIX:PATH=/usr -DGEN_LANGUAGE_BINDINGS=
ON -DCMAKE_BUILD_TYPE:String="Debug" .. && \
    make -j2 && \
    make install && \
    cd ../../cli && mkdir build && cd build && \
    cmake -DCMAKE_INSTALL_PREFIX:PATH=/usr -DGEN_LANGUAGE_BINDINGS=
ON -DCMAKE_BUILD_TYPE:String="Debug" .. && \
    make -j2 && \
    make install

ENV LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/usr/local/lib/IPsec:/usr/lib/
IPsec/

#Install cfgipec2
RUN set -e -x; \
    git clone https://gitlab.atika.um.es/gabilm.um.es/cfgIPsec2.
git; \
    cd cfgIPsec2;

RUN \
    cp /opt/dev/cfgIPsec2/ietf-ipsec.yang /opt/dev/sysrepo/build/
repository && \
    sysrepoctl -i /opt/dev/sysrepo/build/repository/ietf-ipsec.yang

```



```
-e case1

ENV SRC_PATH /home/netconf/cfgIPsec2
WORKDIR $SRC_PATH

COPY supervisord.conf /etc/supervisord.conf
CMD ["/usr/bin/supervisord", "-c", "/etc/supervisord.conf"]

EXPOSE 830
```

Una vez mostrado el fichero que arranca al controlador, mostramos el fichero que inicia los dispositivos a configurar por el controlador, llamado *netopeer-server*.

```
FROM ubuntu:18.04

MAINTAINER mislav.novakovic@sartura.hr

RUN \
    apt-get update && apt-get install -y \
    # general tools
    net-tools \
    git \
    cmake \
    build-essential \
    vim \
    supervisor \
    # libyang
    libpcres-dev \
    pkg-config \
    # sysrepo
    libavl-dev \
    libev-dev \
    libprotobuf-c-dev \
    protobuf-c-compiler \
    # netopeer2 \
    libssh-dev \
    libssl-dev \
    # bindings
    swig \
    python3-dev \
    python3-pip \
    iproute2 tcpdump iputils-ping curl libpam0g-dev autotools-dev
    autoconf strongswan strongswan-swanctl curl doxygen graphviz
    devscripts debhelper rpm bison flex

# add netconf user
RUN \
    adduser --system netconf && \
    echo "netconf:netconf" | chpasswd

# generate ssh keys for netconf user
```

```

RUN \
  mkdir -p /home/netconf/.ssh && \
  ssh-keygen -A && \
  ssh-keygen -t dsa -P '' -f /home/netconf/.ssh/id_dsa && \
  cat /home/netconf/.ssh/id_dsa.pub > /home/netconf/.ssh/
  authorized_keys

# use /opt/dev as working directory
RUN mkdir /opt/dev
WORKDIR /opt/dev

# libyang
RUN \
  git clone https://github.com/CESNET/libyang.git && \
  cd libyang && mkdir build && cd build && \
  git checkout devel && \
  cmake -DCMAKE_INSTALL_PREFIX:PATH=/usr -DGEN_LANGUAGE_BINDINGS=
ON -DCMAKE_BUILD_TYPE:String="Debug" -DENABLE_BUILD_TESTS=OFF ..
  && \
  make -j2 && \
  make install && \
  ldconfig

# sysrepo
RUN \
  git clone https://github.com/sysrepo/sysrepo.git && \
  cd sysrepo && mkdir build && cd build && \
  git checkout devel && \
  cmake -DCMAKE_INSTALL_PREFIX:PATH=/usr -DGEN_LANGUAGE_BINDINGS=
ON -DCMAKE_BUILD_TYPE:String="Debug" -DENABLE_TESTS=OFF -
DREPOSITORY_LOC:PATH=/etc/sysrepo .. && \
  make -j2 && \
  make install && \
  ldconfig

# libnetconf2
RUN \
  git clone https://github.com/CESNET/libnetconf2.git && \
  cd libnetconf2 && mkdir build && cd build && \
  git checkout devel && \
  cmake -DCMAKE_INSTALL_PREFIX:PATH=/usr -DGEN_LANGUAGE_BINDINGS=
ON -DCMAKE_BUILD_TYPE:String="Debug" -DENABLE_BUILD_TESTS=OFF ..
  && \
  make -j2 && \
  make install && \
  ldconfig

# keystore
#RUN \
#  cd /opt/dev && \
#  git clone https://github.com/CESNET/Netopeer2.git && \
#  cd Netopeer2 && \
#  cd keystore && mkdir build && cd build && \

```

```

#     git checkout devel-server && \
#     cmake -DCMAKE_INSTALL_PREFIX:PATH=/usr -DGEN_LANGUAGE_BINDINGS=
ON -DCMAKE_BUILD_TYPE:String="Debug" .. && \
#     make -j2 && \
#     make install && \
#     ldconfig

# netopeer2
RUN \
    cd /opt/dev && \
    git clone https://github.com/CESNET/Netopeer2.git && \
    cd Netopeer2/server && mkdir build && cd build && \
    cmake -DCMAKE_INSTALL_PREFIX:PATH=/usr -DGEN_LANGUAGE_BINDINGS=
ON -DCMAKE_BUILD_TYPE:String="Debug" .. && \
    make -j2 && \
    make install && \
    cd ../../cli && mkdir build && cd build && \
    cmake -DCMAKE_INSTALL_PREFIX:PATH=/usr -DGEN_LANGUAGE_BINDINGS=
ON -DCMAKE_BUILD_TYPE:String="Debug" .. && \
    make -j2 && \
    make install

ENV LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/usr/local/lib/IPsec:/usr/lib/
IPsec/

# strongswan
ARG STRONGSWAN_VERSION=5.5.0
ENV STRONGSWAN_VERSION_BUILD=$STRONGSWAN_VERSION

RUN set -e -x; \
    curl -O https://download.strongswan.org/strongswan-
$STRONGSWAN_VERSION_BUILD.tar.bz2; \
    tar xvf strongswan* strongswan-5.5.0/src/libcharon/plugins/vici/
libvici.h;

RUN set -e -x; cp /opt/dev/strongswan-$STRONGSWAN_VERSION_BUILD/src/
libcharon/plugins/vici/libvici.h /usr/local/include/;

ENV LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/usr/local/lib/IPsec:/usr/lib/
IPsec/

# cfgipsec2
RUN set -e -x; \
    git clone https://gitlab.atika.um.es/gabilm.um.es/cfgIPsec2.git;
\
    cd cfgIPsec2;

RUN \
    cp /opt/dev/cfgIPsec2/ietf-ipsec.yang /opt/dev/sysrepo/build/
repository && \
    sysrepoctl -i /opt/dev/sysrepo/build/repository/ietf-ipsec.yang
-e case1

```

```
# vici
RUN set -e -x; \
    pip3 install vici;

ENV SRC_PATH /home/netconf/cfgIPsec2
WORKDIR $SRC_PATH

COPY supervisord.conf /etc/supervisord.conf
CMD ["/usr/bin/supervisord", "-c", "/etc/supervisord.conf"]

EXPOSE 830
```

Anexo B - Ficheros XML utilizados

En este anexo, vamos a mostrar el fichero XML que hemos utilizado desde el controlador para instanciar la configuración en cada uno de los dispositivos de la topología. En primer lugar mostramos el fichero XML que instanciamos desde el controlador al primer *host*.

```
<ietf-ipsec xmlns="http://example.net/ietf-ipsec" xmlns:nc="urn:ietf:
  params:xml:ns:netconf:base:1.0">
<IPsec nc:operation="merge">
  <spd>
    <spd-entry>
      <rule-number>1</rule-number>
      <names>
        <name>out/192.168.123.100/192.168.123.200</name>
      </names>
      <condition>
        <traffic-selector-list>
          <ts-number>7</ts-number>
          <direction>OUTBOUND</direction>
          <local-addresses>
            <start>192.168.123.100/32</start>
            <end>192.168.123.100/32</end>
          </local-addresses>
          <remote-addresses>
            <start>192.168.123.200/32</start>
            <end>192.168.123.200/32</end>
          </remote-addresses>
          <next-layer-protocol>TCP</next-layer-protocol>
          <local-ports>
            <start>0</start>
            <end>0</end>
          </local-ports>
          <remote-ports>
            <start>0</start>
            <end>0</end>
          </remote-ports>
        </traffic-selector-list>
      </condition>
      <processing-info>
        <action>PROTECT</action>
        <IPsec-sa-cfg>
          <security-protocol>esp</security-protocol>
          <mode>TRANSPORT</mode>
        </IPsec-sa-cfg>
      </processing-info>
    </spd-entry>
  </spd>
  <pad>
    <pad-entries>
      <pad-entry-id>7</pad-entry-id>
      <ipv4-address>192.168.123.200</ipv4-address>
```

```

    <pad-auth-protocol>IKEv2</pad-auth-protocol>
    <auth-method>
      <auth-m>pre-shared</auth-m>
      <pre-shared>
        <secret>secreto_compartido</secret>
      </pre-shared>
    </auth-method>
  </pad-entries>
</pad>
</IPsec>
<ikev2>
  <ike-connection>
    <ike-conn-entries>
      <conn-name>gateway1</conn-name>
      <autostartup>ALWAYSON</autostartup>
      <version>ikev2</version>
      <ike-reauth-lifetime>60</ike-reauth-lifetime>
      <ike-sa-lifetime>30</ike-sa-lifetime>
      <IPsec-sa-lifetime>10</IPsec-sa-lifetime>
      <dh_group>2048</dh_group>
      <local>
        <ipv4>192.168.123.100</ipv4>
        <my-identifier>Ignacio</my-identifier>
      </local>
      <remote>
        <ipv4>192.168.123.200</ipv4>
        <my-identifier>Ruben</my-identifier>
      </remote>
      <pfs_group>0</pfs_group>
      <!--<phase2-lifetime>100</phase2-lifetime>
      <phase2-authalg>hmac-md5-96</phase2-authalg>
      <phase2-encalg>3des</phase2-encalg>-->
    </ike-conn-entries>
  </ike-connection>
</ikev2>
</ietf-ipsec>

```

Acto seguido, mostramos el fichero XML que hemos utilizado para instanciar la configuración correspondiente en el segundo *host*.

```

<ietf-ipsec xmlns="http://example.net/ietf-ipsec" xmlns:nc="urn:ietf:
  params:xml:ns:netconf:base:1.0">
<IPsec>
  <spd>
    <spd-entry>
      <rule-number>1</rule-number>
      <names>
        <name>out/192.168.123.200/192.168.123.100</name>
      </names>
      <condition>
        <traffic-selector-list>
          <ts-number>7</ts-number>

```

```

    <direction>OUTBOUND</direction>
    <local-addresses>
      <start>192.168.123.200/32</start>
      <end>192.168.123.200/32</end>
    </local-addresses>
    <remote-addresses>
      <start>192.168.123.100/32</start>
      <end>192.168.123.100/32</end>
    </remote-addresses>
    <next-layer-protocol>TCP</next-layer-protocol>
    <local-ports>
      <start>0</start>
      <end>0</end>
    </local-ports>
    <remote-ports>
      <start>0</start>
      <end>0</end>
    </remote-ports>
  </traffic-selector-list>
</condition>
<processing-info>
  <action>PROTECT</action>
  <IPsec-sa-cfg>
    <security-protocol>esp</security-protocol>
    <mode>TRANSPORT</mode>
  </IPsec-sa-cfg>
</processing-info>
</spd-entry>
</spd>
<pad>
  <pad-entries>
    <pad-entry-id>7</pad-entry-id>
    <ipv4-address>192.168.123.100</ipv4-address>
    <pad-auth-protocol>IKEv2</pad-auth-protocol>
    <auth-method>
      <auth-m>pre-shared</auth-m>
      <pre-shared>
        <secret>secreto_compartido</secret>
      </pre-shared>
    </auth-method>
  </pad-entries>
</pad>
</IPsec>
<ikev2>
  <ike-connection>
    <ike-conn-entries>
      <conn-name>gateway2</conn-name>
      <autostartup>ALWAYSON</autostartup>
      <version>ikev2</version>
      <ike-reauth-lifetime>0</ike-reauth-lifetime>
      <ike-sa-lifetime>0</ike-sa-lifetime>
      <IPsec-sa-lifetime>0</IPsec-sa-lifetime>
      <dh_group>2048</dh_group>
    </ike-conn-entries>
  </ike-connection>
</ikev2>

```

```
<local>
  <ipv4>192.168.123.200</ipv4>
  <my-identifier>Ruben</my-identifier>
</local>
<remote>
  <ipv4>192.168.123.100</ipv4>
  <my-identifier>Ignacio</my-identifier>
</remote>
<pfs_group>0</pfs_group>
<!--<phase2-lifetime>100</phase2-lifetime>
<phase2-authalg>hmac-md5-96</phase2-authalg>
<phase2-encalg>3des</phase2-encalg>-->
</ike-conn-entries>
</ike-connection>
</ikev2>
</ietf-ipsec>
```

Estos XML corresponden a la primera prueba realizada, en la que se generaba una asociación de seguridad de IPsec en modo transporte. Para la segunda prueba, se realiza usando los mismos XML, donde la única diferencia radica en el cambio del valor del atributo *mode*, donde en este caso se indicaría *TUNNEL* (realmente con omitir el atributo sería suficiente, puesto que por defecto funciona en modo túnel).